

Go (Golang)

SDE2 Interview Guide

100 Questions & Answers with Code Examples

Level: Mid (2–4 yrs)	Stack: Go / Golang	Roles: SDE2 / Backend
----------------------	--------------------	-----------------------

#	Topic	Questions
1	Goroutines & Concurrency	Q1–Q15
2	Channels & Select	Q16–Q28
3	Runtime & Memory	Q29–Q40
4	Interfaces & Types	Q41–Q52
5	Error Handling	Q53–Q60
6	Context	Q61–Q67
7	Generics	Q68–Q74
8	Testing & Benchmarking	Q75–Q82
9	Go Patterns	Q83–Q92
10	Miscellaneous & Advanced	Q93–Q100

> **Focus:** Architecture, Internals, Indexing, Transactions, Replication, Go Integration | **Level:** SDE2

Table of Contents

[Data Types & Schema Design](#2-data-types--schema-design) — Q26–Q50
 [Indexing & Query Optimization](#3-indexing--query-optimization) — Q51–Q90
 [Transactions & Concurrency](#4-transactions--concurrency) — Q91–Q120
 [Replication & High Availability](#5-replication--high-availability) — Q121–Q145
 [Partitioning & Sharding](#6-partitioning--sharding) — Q146–Q160
 [Advanced Features & Operations](#7-advanced-features--operations) — Q161–Q185
 [Go + PostgreSQL Patterns](#8-go--postgresql-patterns) — Q186–Q200

Core Architecture & Internals

Q1. What is PostgreSQL's process model?

Difficulty:
Medium

PostgreSQL uses **multi-process** (not multi-threaded). One postmaster forks a backend process per connection (~5 MB RAM each). Background workers: autovacuum, bgwriter, walwriter, checkpoint, stats collector.

```
postmaster
■■■ backend (pid 101) ← client 1
■■■ backend (pid 102) ← client 2
■■■ autovacuum launcher
■■■ bgwriter
■■■ walwriter
■■■ checkpoint
```

***Interview tip:** "Each connection = 1 OS process ≈ 5 MB. 200 connections = 1 GB overhead. This is why PgBouncer is mandatory at scale."*

Q2. What is the shared buffer pool?

Difficulty:
Medium

Shared buffers is PostgreSQL's in-memory page cache (8 KB pages). Set to 25–40% of RAM. Clock-sweep replacement policy (approximate LRU).

```
-- Check cache hit rate (should be >99%)
SELECT round(
  sum(blks_hit)::numeric /
  nullif(sum(blks_hit)+sum(blks_read),0) * 100, 2
) AS hit_ratio
FROM pg_stat_database;

-- Per-table
SELECT relname,
  heap_blks_hit,
  heap_blks_read,
  round(heap_blks_hit::numeric /
    nullif(heap_blks_hit+heap_blks_read,0)*100,1) AS hit_pct
FROM pg_statio_user_tables
ORDER BY heap_blks_read DESC LIMIT 10;
```

Q3. What is MVCC?**Difficulty: Hard**

Multi-Version Concurrency Control: readers never block writers; writers never block readers. Each row has xmin (created-by TX) and xmax (deleted-by TX). Readers see a snapshot at transaction start.

```
-- Inspect hidden MVCC columns
SELECT *, xmin, xmax, ctid FROM orders LIMIT 5;
-- ctid = (page_number, tuple_offset)
-- xmax=0 means the row is visible (not deleted)
```

Interview tip: "MVCC is why UPDATE creates a new row version instead of modifying in place. Old versions accumulate → VACUUM reclaims them."

Q4. What is VACUUM and why is it needed?**Difficulty: Medium**

VACUUM reclaims storage from dead row versions (MVCC UPDATE/DELETE leftovers). Also updates visibility maps and advances the oldest transaction ID (prevents wraparound).

```
VACUUM ANALYZE orders;
VACUUM FULL orders; -- reclaims disk space; avoid in prod (use pg_repack)

-- Tables needing vacuum
SELECT relname, n_dead_tup, last_autovacuum,
       round(n_dead_tup::numeric/nullif(n_live_tup,0)*100,1) AS dead_pct
FROM pg_stat_user_tables
WHERE n_dead_tup > 1000
ORDER BY n_dead_tup DESC;
```

Q5. What is autovacuum and how do you tune it?**Difficulty: Hard**

Autovacuum automatically runs VACUUM/ANALYZE. Key parameters: autovacuum_vacuum_scale_factor (default 0.2 = 20% dead tuples triggers vacuum), autovacuum_naptime, autovacuum_max_workers.

```
-- Per-table tuning for high-write tables
ALTER TABLE orders SET (
    autovacuum_vacuum_scale_factor = 0.01, -- trigger at 1% dead tuples
    autovacuum_vacuum_threshold = 100,
    autovacuum_vacuum_cost_delay = 2
);

-- Watch autovacuum
SELECT pid, query, state FROM pg_stat_activity
WHERE query LIKE 'autovacuum%';
```

Interview tip: "Default 20% scale factor = 1M-row table needs 200K dead tuples before autovacuum runs. For OLTP: set 1–5%."

Q6. What is the WAL (Write-Ahead Log)?**Difficulty: Medium**

WAL records every change before applying to data files. On crash: replay WAL from last checkpoint. Sequential writes → 10–100× faster than random data-file writes. Also enables streaming replication and PITR.

```
SELECT pg_current_wal_lsn(); -- current WAL position
SELECT pg_size_pretty(sum(size)) FROM pg_ls_waldir(); -- WAL size

SHOW wal_level; -- minimal | replica | logical
```

```
SHOW synchronous_commit; -- on | off | remote_write | remote_apply
```

Q7. What is a checkpoint?

Difficulty:
Medium

Checkpoint flushes all dirty buffer pages to disk and records the checkpoint LSN. On crash, only WAL after the last checkpoint needs replaying.

```
CHECKPOINT; -- force manual checkpoint
```

```
SELECT checkpoints_timed, checkpoints_req,  
       buffers_checkpoint, buffers_backend,  
       checkpoint_write_time, checkpoint_sync_time  
FROM pg_stat_bgwriter;  
-- buffers_backend > 0 = checkpoints too infrequent (increase max_wal_size)
```

Q8. What is TOAST?

Difficulty:
Medium

TOAST (The Oversized-Attribute Storage Technique): values wider than ~2 KB are automatically compressed and/or stored in a separate TOAST table. Transparent to queries.

```
-- Check TOAST strategy per column  
SELECT attname, attstorage -- p=plain, e=extended, x=external, m=main  
FROM pg_attribute  
WHERE attrelid = 'orders'::regclass AND attnum > 0;
```

```
ALTER TABLE posts ALTER COLUMN body SET STORAGE external;  
-- external = move out, no compress (faster access, larger storage)
```

Q9. What is the query planner and how does it pick plans?

Difficulty: Hard

Planner generates candidate plans, estimates cost = (pages × I/O cost) + (rows × CPU cost), picks cheapest. Uses statistics from pg_stats. Cost parameters: seq_page_cost=1.0, random_page_cost=4.0 (SSD: set 1.1).

```
EXPLAIN (ANALYZE, BUFFERS)  
SELECT * FROM orders o  
JOIN users u ON o.user_id = u.id  
WHERE o.created_at > NOW() - INTERVAL '7 days';  
  
-- Key terms:  
-- Seq Scan: full table scan  
-- Index Scan: B-tree → heap fetch per row  
-- Index Only Scan: all data in index, no heap  
-- Hash Join: build hash from smaller table  
-- Nested Loop: for each outer row, index-lookup inner
```

Q10. What is the difference between Seq Scan, Index Scan, Bitmap Scan?

Difficulty:
Medium

Seq Scan: reads all pages sequentially. Best for large fraction of table. **Index Scan:** B-tree walk + random heap fetch per row. Best for tiny result sets. **Bitmap Index Scan:** collect matching pages into bitmap, sort, fetch heap sequentially. Best for moderate sets.

```
-- Tune for SSD (reduces planner preference for seq scan)  
SET random_page_cost = 1.1;  
SET effective_io_concurrency = 200;
```

Q11. What are pg_stat_statements and how do you use them?**Difficulty:**
Medium

```
CREATE EXTENSION pg_stat_statements;

-- Top 10 queries by total time
SELECT substring(query,1,80) AS query, calls,
       round(mean_exec_time::numeric,2) AS mean_ms,
       round(total_exec_time::numeric/1000,2) AS total_sec,
       round(100.0*shared_blks_hit/
       nullif(shared_blks_hit+shared_blks_read,0),1) AS hit_pct
FROM pg_stat_statements
ORDER BY total_exec_time DESC LIMIT 10;

SELECT pg_stat_statements_reset(); -- reset stats
```

Interview tip: "Install pg_stat_statements on every production PostgreSQL. Set `pg\_stat\_statements.max=10000`. It's the single most useful performance tool."

Q12. What is pg_stat_activity and how do you detect blocking?**Difficulty:**
Medium

```
-- Long-running queries
SELECT pid, now()-query_start AS duration, state, wait_event_type, query
FROM pg_stat_activity
WHERE state != 'idle' AND query_start < NOW()-INTERVAL '1 min'
ORDER BY duration DESC;

-- Who is blocking whom
SELECT blocked.pid, blocked.query, blocking.pid AS blocker_pid, blocking.query
FROM pg_stat_activity blocked
JOIN pg_stat_activity blocking
  ON blocking.pid = ANY(pg_blocking_pids(blocked.pid))
WHERE cardinality(pg_blocking_pids(blocked.pid)) > 0;

SELECT pg_cancel_backend(pid);    -- SIGINT (graceful)
SELECT pg_terminate_backend(pid); -- SIGTERM (force)
```

Q13. What is the cost model and how do you tune it for SSDs?**Difficulty: Hard**

```
-- Default values assume spinning disk:
SHOW seq_page_cost;      -- 1.0
SHOW random_page_cost;   -- 4.0 (too high for SSD!)
SHOW effective_cache_size; -- ~4GB (should be 75% of RAM)

-- Tune for NVMe SSD:
ALTER SYSTEM SET random_page_cost = 1.1;
ALTER SYSTEM SET effective_io_concurrency = 200;
ALTER SYSTEM SET effective_cache_size = '12GB'; -- 75% of 16GB RAM
SELECT pg_reload_conf();
```

Interview tip: "Wrong `random\_page\_cost` is the #1 misconfiguration on cloud servers. Default 4.0 makes planner avoid index scans that would actually be fast on SSD."

Q14. What is the pg_catalog and information_schema?**Difficulty: Easy**

pg_catalog: PostgreSQL system tables (pg_class, pg_attribute, pg_index). Lower-level but richer and faster. **information_schema:** SQL-standard views, portable across RDBMS but slower.

```
-- pg_catalog (faster, more detail):
SELECT c.relname, a.attname, t.typname
FROM pg_class c
JOIN pg_attribute a ON a.attrelid = c.oid
JOIN pg_type t ON t.oid = a.atttypid
WHERE c.relname = 'orders' AND a.attnum > 0;

-- information_schema (portable):
SELECT column_name, data_type FROM information_schema.columns
WHERE table_name = 'orders';
```

Q15. What are tablespaces?

Difficulty: Easy

Tablespaces map logical names to filesystem directories. Place hot data on NVMe SSD, cold data on HDD.

```
CREATE TABLESPACE fast LOCATION '/mnt/nvme/pgdata';
CREATE TABLE hot_events (...) TABLESPACE fast;
ALTER INDEX idx_events_user SET TABLESPACE fast;
SELECT spcname, pg_tablespace_location(oid) FROM pg_tablespace;
```

Q16. What is transaction ID wraparound and how do you prevent it?

Difficulty: Hard

TX IDs are 32-bit (~4 billion). Tables must be vacuumed before they age 2 billion TXs. VACUUM FREEZE marks rows as "frozen" (eternally visible).

```
-- Check database age
SELECT datname, age(datfrozenxid),
       2000000000 - age(datfrozenxid) AS txns_remaining
FROM pg_database ORDER BY age(datfrozenxid) DESC;
-- Alert if age > 1.5 billion!

VACUUM FREEZE VERBOSE orders;

-- autovacuum_freeze_max_age = 200M (trigger freeze at this age)
ALTER TABLE orders SET (autovacuum_freeze_max_age = 150000000);
```

Interview tip: "Transaction ID wraparound = PostgreSQL shuts down and refuses connections. Monitor `age(datfrozenxid)`. Alert at 1.2B, emergency at 1.8B."

Q17. What is EXPLAIN (ANALYZE, BUFFERS) and how do you read it?

Difficulty: Hard

```
EXPLAIN (ANALYZE, BUFFERS, FORMAT TEXT)
SELECT u.name, COUNT(o.id)
FROM users u LEFT JOIN orders o ON o.user_id = u.id
WHERE u.created_at > '2024-01-01'
GROUP BY u.id ORDER BY 2 DESC LIMIT 10;
```

/*

Key things to check:

1. Actual rows vs Estimated rows → large diff = stale statistics → ANALYZE
2. Node type: Seq Scan on large table = missing index
3. Buffers: shared read=900 hit=100 → mostly disk I/O (cold cache / large table)
4. Sort Method: "external merge Disk: 1234kB" → work_mem too low
5. Loops: actual rows=100 loops=500 → 50K total fetches

*/

Q18. What is pg_stat_bgwriter and why does it matter?**Difficulty:**
Medium

```
SELECT checkpoints_timed, checkpoints_req,
       buffers_checkpoint, buffers_clean, buffers_backend
FROM pg_stat_bgwriter;

-- buffers_backend > 0 = app backends writing dirty buffers (bad!)
-- Fix: increase max_wal_size so checkpoints are less frequent
-- checkpoints_req >> checkpoints_timed = max_wal_size too small
```

Q19. What are PostgreSQL roles and privileges?**Difficulty: Easy**

```
CREATE ROLE app_user WITH LOGIN PASSWORD 'pass';
GRANT CONNECT ON DATABASE myapp TO app_user;
GRANT USAGE ON SCHEMA public TO app_user;
GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA public TO app_user;
GRANT USAGE, SELECT ON ALL SEQUENCES IN SCHEMA public TO app_user;

-- Row-level security
ALTER TABLE orders ENABLE ROW LEVEL SECURITY;
CREATE POLICY tenant_policy ON orders
  USING (tenant_id = current_setting('app.tenant_id')::bigint);
```

Q20. What are the most important PostgreSQL config parameters?**Difficulty:**
Medium

```
shared_buffers = 4GB           # 25% of RAM
effective_cache_size = 12GB     # 75% of RAM (planner hint)
work_mem = 64MB                 # per sort/hash op (careful!)
maintenance_work_mem = 1GB     # VACUUM, CREATE INDEX
checkpoint_completion_target = 0.9
max_wal_size = 4GB
random_page_cost = 1.1         # SSD
effective_io_concurrency = 200  # SSD
max_connections = 200          # use PgBouncer for more
idle_in_transaction_session_timeout = '5min'
log_min_duration_statement = 1000 # log slow queries >1s
autovacuum_vacuum_scale_factor = 0.05
```

Q21. What is connection pooling and how does PgBouncer work?**Difficulty:**
Medium

```
# pgbouncer.ini
[databases]
myapp = host=postgres port=5432 dbname=myapp

[pgbouncer]
pool_mode = transaction          # most efficient
max_client_conn = 10000
default_pool_size = 25
server_idle_timeout = 600

// Connect to PgBouncer (port 6432), not PostgreSQL (5432)
dsn := "postgres://user:pass@pgbouncer:6432/myapp"
db.SetMaxOpenConns(50)
```

Interview tip: "Transaction pooling: connection returned to pool after each transaction. Caveat: SET, PREPARE don't persist — use SET LOCAL inside transactions."

Q22. What is pg_dump and how do you use it in production?

Difficulty: Easy

```
pg_dump -Fc -d myapp -f backup.dump      # custom format (compressed)
pg_dump -Fd -j 4 -d myapp -f backup_dir/ # parallel dump
pg_restore -d newdb -j 4 backup.dump     # parallel restore
pg_restore -d myapp -t orders backup.dump # restore single table

# PITR: enable WAL archiving
# archive_mode = on
# archive_command = 'aws s3 cp %p s3://backups/wal/%f'
```

Q23. What is hot standby feedback?

Difficulty: Hard

```
-- hot_standby_feedback = on
-- Tells primary about long-running standby queries
-- Primary won't vacuum away rows still needed by standby
-- Trade-off: may cause table bloat on primary

SHOW hot_standby_feedback;

-- max_standby_streaming_delay = 30s
-- How long standby waits before canceling conflicting query
-- -1 = wait forever
```

Q24. What is pg_repack and when do you use it?

Difficulty: Medium

```
# pg_repack: online table/index rebuild without heavy locks
# Use instead of VACUUM FULL (which needs ACCESS EXCLUSIVE lock)

pg_repack --table=orders mydb
pg_repack --index=idx_orders_user_id mydb
pg_repack --dry-run mydb
```

Interview tip: "VACUUM FULL blocks all reads and writes. pg_repack rebuilds the table online using triggers to track changes during rebuild. Use whenever bloat >30%."

Q25. What is PITR (Point-In-Time Recovery)?

Difficulty: Medium

```
# PITR = base backup + continuous WAL archiving
# Lets you restore to any past point in time

# postgresql.conf:
# archive_mode = on
# archive_command = 'cp %p /archive/%f'

# Restore:
# restore_command = 'cp /archive/%f %p'
# recovery_target_time = '2024-01-15 14:30:00'

# Check recovery status
```



```
SELECT pg_is_in_recovery(), pg_last_xact_replay_timestamp();
```

Data Types & Schema Design

Q26. What numeric types should you use?

Difficulty: Easy

```
-- Whole numbers
SMALLINT / INT2          -- 2 bytes, ±32K
INTEGER / INT4           -- 4 bytes, ±2B ← use for most IDs
BIGINT / INT8            -- 8 bytes, ±9.2×10^18

-- Decimals
NUMERIC(p,s)            -- exact, slow (use for money!)
FLOAT4 / REAL            -- 4 bytes approx
FLOAT8 / DOUBLE          -- 8 bytes approx

-- Modern primary key (replaces SERIAL)
id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY

-- NEVER use float for money:
SELECT 0.1::float + 0.2::float = 0.3; -- FALSE
SELECT 0.1::numeric + 0.2::numeric = 0.3::numeric; -- TRUE
```

Q27. What is JSONB vs JSON?

Difficulty: Medium

JSON: stores raw text, preserves key order, fast write, slow read. JSONB: binary format, deduplicates keys, slower write, fast read, supports GIN indexing. **Always use JSONB.**

```
CREATE TABLE events (
  id BIGINT GENERATED ALWAYS AS IDENTITY,
  payload JSONB NOT NULL
);

-- Operators
SELECT payload->>'user_id',          -- text result
       payload->'metadata'->'source' -- jsonb result
FROM events
WHERE payload @> '{"event_type":"click"}'; -- containment (uses GIN index)

CREATE INDEX idx_events_gin ON events USING GIN(payload);
CREATE INDEX idx_events_fast ON events USING GIN(payload jsonb_path_ops);
-- jsonb_path_ops: smaller, faster for @> but not ?
```

Q28. How do you handle arrays?

Difficulty: Medium

```
CREATE TABLE articles (id INT, tags TEXT[], scores INT[]);
INSERT INTO articles VALUES (1, ARRAY['go','backend'], ARRAY[90,85]);

SELECT * FROM articles WHERE tags @> ARRAY['go']; -- contains
SELECT * FROM articles WHERE 'go' = ANY(tags);    -- equivalent
SELECT * FROM articles WHERE tags && ARRAY['go','rust']; -- overlap
SELECT unnest(tags) FROM articles;                 -- flatten
```

```
CREATE INDEX idx_tags ON articles USING GIN(tags);

UPDATE articles SET tags = array_append(tags, 'k8s') WHERE id=1;
UPDATE articles SET tags = array_remove(tags, 'go') WHERE id=1;
```

Q29. When should you use UUID vs BIGINT for primary keys?

Difficulty:
Medium

```
-- BIGINT IDENTITY: sequential, 8 bytes, B-tree friendly
id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY

-- UUID v4: globally unique, 16 bytes, RANDOM → index fragmentation
id UUID DEFAULT gen_random_uuid() PRIMARY KEY

-- UUID causes B-tree page splits (random inserts → 50% full pages)
-- Use UUID only if: global uniqueness across systems, or exposing IDs externally
-- For internal tables: BIGINT IDENTITY is almost always better
```

Q30. What are partial indexes?

Difficulty:
Medium

```
-- Index only active (non-deleted) records
CREATE INDEX idx_users_email_active ON users(email)
WHERE deleted_at IS NULL;

-- Index only recent orders (most queries are recent)
CREATE INDEX idx_orders_recent ON orders(created_at, user_id)
WHERE created_at > NOW() - INTERVAL '90 days';

-- Partial unique: unique among non-deleted
CREATE UNIQUE INDEX idx_users_email_uniq
ON users(email) WHERE deleted_at IS NULL;
```

Interview tip: "Partial indexes can be 10–100× smaller than full indexes. Always add on soft-delete patterns."

Q31. What are expression (functional) indexes?

Difficulty:
Medium

```
-- Case-insensitive email lookup
CREATE INDEX idx_users_email_lower ON users(lower(email));
-- Query must use same expression:
SELECT * FROM users WHERE lower(email) = lower($1);

-- JSON field extraction
CREATE INDEX idx_events_type ON events((payload->>'event_type'));
SELECT * FROM events WHERE payload->>'event_type' = 'click';

-- Expression must match exactly or index won't be used
```

Q32. What is the RETURNING clause?

Difficulty: Easy

```
-- Get generated ID after INSERT (no extra SELECT)
INSERT INTO orders (user_id, amount)
VALUES (42, 99.99)
RETURNING id, created_at;

-- Get updated rows
```

```
UPDATE orders SET status='shipped', shipped_at=NOW()
WHERE id=123 RETURNING id, status, shipped_at;

-- In Go:
var id int64
var createdAt time.Time
db.QueryRowContext(ctx,
    "INSERT INTO orders (user_id,amount) VALUES ($1,$2) RETURNING id,created_at",
    userID, amount,
).Scan(&id, &createdAt)
```

Q33. What is INSERT ... ON CONFLICT (upsert)?

Difficulty:
Medium

```
-- Upsert by primary key
INSERT INTO users (id, email, name)
VALUES (42, 'alice@ex.com', 'Alice')
ON CONFLICT (id) DO UPDATE
    SET email = EXCLUDED.email, name = EXCLUDED.name, updated_at = NOW();

-- Counter increment (atomic)
INSERT INTO counters (key, value) VALUES ('views', 1)
ON CONFLICT (key) DO UPDATE SET value = counters.value + EXCLUDED.value;

-- Ignore conflicts
INSERT INTO events (id, data) VALUES ($1, $2)
ON CONFLICT DO NOTHING;

-- EXCLUDED = the row that would have been inserted
```

Q34. What are materialized views?

Difficulty:
Medium

```
CREATE MATERIALIZED VIEW daily_revenue AS
    SELECT date_trunc('day',created_at)::DATE AS day,
           SUM(amount) AS revenue, COUNT(*) AS orders
    FROM orders WHERE status='paid'
    GROUP BY 1 ORDER BY 1;

-- Unique index required for CONCURRENTLY
CREATE UNIQUE INDEX idx_daily_revenue_day ON daily_revenue(day);

-- Refresh without blocking reads
REFRESH MATERIALIZED VIEW CONCURRENTLY daily_revenue;

-- Schedule with pg_cron:
SELECT cron.schedule('refresh-revenue','*/15 * * * *',
    'REFRESH MATERIALIZED VIEW CONCURRENTLY daily_revenue');
```

Q35. What are window functions?

Difficulty: **Hard**

```
SELECT user_id, order_id, amount,
       ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY amount DESC) AS rn,
       RANK() OVER (PARTITION BY user_id ORDER BY amount DESC) AS rnk,
       SUM(amount) OVER (PARTITION BY user_id) AS user_total,
       LAG(amount) OVER (PARTITION BY user_id ORDER BY created_at) AS prev_amt,
```

```

    LEAD(amount) OVER (PARTITION BY user_id ORDER BY created_at) AS next_amt
FROM orders;

-- Top order per user
SELECT * FROM (
    SELECT *, ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY amount DESC) AS rn
    FROM orders
) t WHERE rn = 1;

-- Running total
SELECT created_at, amount,
    SUM(amount) OVER (ORDER BY created_at
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS running_total
FROM orders;
```

Q36. What are CTEs (Common Table Expressions)?

Difficulty:
Medium

```

-- Simple CTE for readability
WITH monthly AS (
    SELECT date_trunc('month',created_at) AS month, SUM(amount) AS rev
    FROM orders WHERE status='paid' GROUP BY 1
), avg_rev AS (
    SELECT AVG(rev) AS avg FROM monthly
)
SELECT month, rev, rev - avg AS diff FROM monthly, avg_rev;

-- Recursive CTE: organizational hierarchy
WITH RECURSIVE org AS (
    SELECT id, name, manager_id, 0 AS depth
    FROM employees WHERE manager_id IS NULL
    UNION ALL
    SELECT e.id, e.name, e.manager_id, o.depth+1
    FROM employees e JOIN org o ON e.manager_id = o.id
)
SELECT * FROM org ORDER BY depth, name;

-- Force materialisation (PG12+ inlines by default)
WITH expensive AS MATERIALIZED (SELECT ...) SELECT * FROM expensive;
```

Q37. What is the difference between WHERE and HAVING?

Difficulty: Easy

```

-- WHERE: before grouping (uses indexes)
-- HAVING: after grouping (no index possible)
SELECT status, COUNT(*) FROM orders
WHERE created_at > NOW()-INTERVAL '30 days' -- filter rows first
GROUP BY status
HAVING COUNT(*) > 100; -- filter groups after

-- Mistake: filter in HAVING instead of WHERE
-- BAD: GROUP BY all 10M rows, then HAVING filters 99% away
-- GOOD: WHERE filters first, GROUP BY small result set
```

Q38. What are generated columns?

Difficulty:
Medium

```
CREATE TABLE products (
  id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  price NUMERIC(10,2) NOT NULL,
  tax_rate NUMERIC(5,4) NOT NULL DEFAULT 0.08,
  price_with_tax NUMERIC(10,2)
    GENERATED ALWAYS AS (price*(1+tax_rate)) STORED,
  name TEXT, description TEXT,
  fts TSVECTOR
    GENERATED ALWAYS AS (
      to_tsvector('english', coalesce(name,'') || ' ' || coalesce(description,''))
    ) STORED
);

CREATE INDEX idx_products_fts ON products USING GIN(fts);
```

Q39. What are row-level security (RLS) policies?**Difficulty: Hard**

```
ALTER TABLE orders ENABLE ROW LEVEL SECURITY;
ALTER TABLE orders FORCE ROW LEVEL SECURITY; -- even for table owner

CREATE POLICY tenant_isolation ON orders
  AS PERMISSIVE FOR ALL TO app_role
  USING (tenant_id = current_setting('app.tenant_id')::bigint)
  WITH CHECK (tenant_id = current_setting('app.tenant_id')::bigint);

-- In Go: set context before queries
db.Exec("SET LOCAL app.tenant_id = $1", tenantID)
-- OR: set on connection acquire
```

Interview tip: "RLS is the safest multi-tenant architecture — even buggy application code can't leak tenant data."

Q40. What are composite types?**Difficulty: Medium**

```
CREATE TYPE address AS (
  street TEXT, city TEXT, country TEXT, zip TEXT
);

CREATE TABLE users (
  id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  name TEXT,
  home_address address,
  work_address address
);

INSERT INTO users (name, home_address)
VALUES ('Alice', ROW('123 Main', 'NYC', 'US', '10001'));
SELECT (home_address).city FROM users;
```

Q41. What are enum types and their limitations?**Difficulty: Easy**

```
CREATE TYPE order_status AS ENUM ('pending','processing','shipped','delivered','cancelled');
CREATE TABLE orders (id BIGINT, status order_status DEFAULT 'pending');

-- Add value (safe)
ALTER TYPE order_status ADD VALUE 'returned' AFTER 'delivered';

-- Cannot remove values without recreation!
```

```
-- Alternative: TEXT + CHECK constraint (more flexible)
status TEXT CHECK (status IN ('pending','processing','shipped'))
```

Q42. What are deferred constraints?

Difficulty: Hard

```
-- Normally: constraints checked after each statement
-- DEFERRABLE: delay checking until COMMIT

ALTER TABLE order_items ADD CONSTRAINT fk_order
    FOREIGN KEY (order_id) REFERENCES orders(id)
    DEFERRABLE INITIALLY DEFERRED; -- checked at COMMIT

-- Circular FK: insert child before parent
BEGIN;
INSERT INTO orgs VALUES (2, 1); -- references id=1 not yet inserted
INSERT INTO orgs VALUES (1, NULL); -- root
COMMIT; -- constraint checked here; both rows exist now

-- Defer normally-immediate constraint for this transaction
SET CONSTRAINTS fk_user DEFERRED;
```

Q43. What is schema design for soft deletes?

Difficulty: Easy

```
CREATE TABLE users (
    id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    email TEXT NOT NULL,
    deleted_at TIMESTAMPTZ -- NULL = active
);

-- Partial unique: unique among active users only
CREATE UNIQUE INDEX idx_users_email ON users(email)
    WHERE deleted_at IS NULL;

-- View for active records
CREATE VIEW active_users AS SELECT * FROM users WHERE deleted_at IS NULL;

-- Soft delete
UPDATE users SET deleted_at = NOW() WHERE id = $1;
-- Purge old records
DELETE FROM users WHERE deleted_at < NOW()-INTERVAL '90 days';
```

Q44. What is efficient pagination in PostgreSQL?

Difficulty: Medium

```
-- OFFSET pagination: SLOW for large offsets
SELECT * FROM orders ORDER BY created_at DESC LIMIT 20 OFFSET 1000000;
-- Scans 1M rows to skip them!

-- Keyset / cursor pagination: O(log N) with index
-- First page:
SELECT * FROM orders ORDER BY created_at DESC, id DESC LIMIT 20;

-- Next page (cursor = last row's created_at + id):
SELECT * FROM orders
WHERE (created_at, id) < ($1::timestamptz, $2::bigint)
ORDER BY created_at DESC, id DESC
```

```
LIMIT 20;

-- Index to support this
CREATE INDEX idx_orders_keyset ON orders(created_at DESC, id DESC);
```

Q45. What is NOT IN vs NOT EXISTS and the NULL trap?**Difficulty:**
Medium

```
-- NOT IN NULL TRAP: if subquery returns ANY NULL → result is empty!
SELECT * FROM users
WHERE id NOT IN (SELECT user_id FROM banned);
-- If banned has even one NULL user_id → returns NO rows!

-- Safe: NOT EXISTS
SELECT * FROM users u
WHERE NOT EXISTS (SELECT 1 FROM banned WHERE user_id = u.id);

-- Safe: LEFT JOIN anti-join
SELECT u.* FROM users u
LEFT JOIN banned b ON b.user_id = u.id
WHERE b.user_id IS NULL;
```

Q46. What is DISTINCT vs DISTINCT ON?**Difficulty:**
Medium

```
-- DISTINCT: remove duplicate rows (all columns)
SELECT DISTINCT user_id, status FROM orders;

-- DISTINCT ON: keep first row per group (PostgreSQL extension)
-- Must match ORDER BY prefix
SELECT DISTINCT ON (user_id) user_id, amount, created_at
FROM orders
ORDER BY user_id, created_at DESC; -- most recent order per user
-- Much faster than: SELECT * FROM orders o WHERE created_at =
-- (SELECT MAX(created_at) FROM orders WHERE user_id = o.user_id)
```

Q47. What is the LATERAL keyword?**Difficulty: Hard**

```
-- LATERAL: subquery can reference preceding FROM items
-- Top 3 orders per user
SELECT u.id, u.name, o.id AS order_id, o.amount
FROM users u,
LATERAL (
    SELECT id, amount FROM orders
    WHERE user_id = u.id -- references u.id
    ORDER BY amount DESC LIMIT 3
) o;

-- LATERAL with unnest
SELECT u.id, tag
FROM users u, LATERAL unnest(u.interests) AS tag;
```

Q48. What are range types?**Difficulty:**
Medium

```
CREATE TABLE reservations (
```

```

    id BIGINT GENERATED ALWAYS AS IDENTITY,
    room_id INT,
    period DATERANGE
);

CREATE INDEX idx_reservations ON reservations USING GIST(period);

-- Exclusion constraint: no overlapping reservations
CREATE EXTENSION btree_gist;
ALTER TABLE reservations ADD CONSTRAINT no_overlap
    EXCLUDE USING GIST (room_id WITH =, period WITH &&);

-- Range operators
SELECT * FROM reservations
WHERE period @> '2024-01-12'::date;    -- contains date
WHERE period && '[2024-01-10,2024-01-20)'::daterange; -- overlaps

```

Q49. What is batch DELETE strategy for large tables?

Difficulty:
Medium

```

-- Never delete millions of rows in one statement
-- Creates massive WAL, holds locks, causes VACUUM pressure

-- Batch delete with sleep
DO $$
BEGIN
    LOOP
        DELETE FROM events WHERE id IN (
            SELECT id FROM events
            WHERE created_at < NOW()-INTERVAL '90 days'
            LIMIT 10000
        );
        EXIT WHEN NOT FOUND;
        PERFORM pg_sleep(0.1); -- breathing room
    END LOOP;
END;
$$;

-- Or with partition DROP (instant, no VACUUM needed)
-- DROP TABLE events_2023_01; -- instant!

```

Q50. What is TRUNCATE vs DELETE?

Difficulty: Easy

```

-- TRUNCATE: drops all pages (instant, no row-level WAL)
-- Requires ACCESS EXCLUSIVE lock, but transactional in PostgreSQL!
TRUNCATE TABLE staging;
TRUNCATE TABLE orders, order_items;
TRUNCATE TABLE users RESTART IDENTITY;

BEGIN;
TRUNCATE TABLE staging;
ROLLBACK; -- works! TRUNCATE is transactional

-- DELETE: generates dead tuple per row, activates triggers, supports WHERE
DELETE FROM orders WHERE created_at < NOW()-INTERVAL '1 year';
-- Requires VACUUM after large delete

```



```
-- Truncate is 100-1000× faster for removing all rows
```

Indexing & Query Optimization

Q51. How does a B-Tree index work?

Difficulty:
Medium

B-Tree: balanced, sorted tree. $O(\log N)$ lookup. Supports: =, <, >, BETWEEN, ORDER BY, IS NULL. Height $\approx 3-4$ for most tables. Leaf pages contain (key, heap ctid) pairs.

```
CREATE INDEX idx_orders_user ON orders(user_id);
CREATE INDEX idx_orders_user_date ON orders(user_id, created_at);
```

```
-- Index health
SELECT indexrelname, idx_scan, idx_tup_read, idx_tup_fetch,
       pg_size_pretty(pg_relation_size(indexrelid)) AS size
FROM pg_stat_user_indexes
WHERE relname = 'orders' ORDER BY idx_scan DESC;
```

```
-- Find unused indexes
SELECT indexrelname, idx_scan
FROM pg_stat_user_indexes WHERE idx_scan = 0
ORDER BY pg_relation_size(indexrelid) DESC;
```

Q52. What is a GIN index?

Difficulty: Hard

GIN (Generalized Inverted Index): each element $\rightarrow$ posting list of row IDs. Slower to build/update than B-Tree. Fast for containment, key-existence queries.

```
-- JSONB containment
CREATE INDEX idx_events_gin ON events USING GIN(payload);
SELECT * FROM events WHERE payload @> '{"type":"click"}';

-- Full-text search
CREATE INDEX idx_articles_fts ON articles USING GIN(
    to_tsvector('english', title||' '||body)
);
SELECT * FROM articles
WHERE to_tsvector('english',title||' '||body)
@@ to_tsquery('english','postgres & index');

-- Arrays
CREATE INDEX idx_products_tags ON products USING GIN(tags);
SELECT * FROM products WHERE tags @> ARRAY['go','backend'];

-- Trigram (LIKE queries)
CREATE EXTENSION pg_trgm;
CREATE INDEX idx_name_trgm ON users USING GIN(name gin_trgm_ops);
SELECT * FROM users WHERE name ILIKE '%alice%'; -- uses index!
```

Q53. What is a GiST index?

Difficulty: Hard

GiST (Generalized Search Tree): extensible, lossy (heap recheck needed). Faster updates than GIN. Used for geometry, ranges, full-text (frequent updates), IP ranges.

```
-- Range type overlap queries
CREATE INDEX idx_bookings ON reservations USING GIST(period);
SELECT * FROM reservations WHERE period && '[2024-01-10,2024-01-20)';

-- PostGIS geometry
CREATE INDEX idx_geo ON locations USING GIST(coords);
SELECT * FROM locations WHERE coords <-> POINT(40.7,-74.0) < 10;

-- GIN vs GiST for FTS:
-- GIN: faster queries, slower updates
-- GiST: slower queries, faster updates
-- Use GIN unless writes are very frequent
```

Q54. What is a BRIN index?

Difficulty:
Medium

BRIN (Block Range Index): stores min/max per block range. Tiny (kilobytes vs gigabytes). Only effective when column values correlate with physical storage order.

```
-- Append-only time-series: created_at always increases → perfect for BRIN
CREATE INDEX idx_logs_brin ON logs USING BRIN(created_at)
  WITH (pages_per_range = 128);

-- Index is 1000x smaller than B-Tree for correlated data
-- SELECT * FROM logs WHERE created_at BETWEEN $1 AND $2 uses BRIN

-- When NOT to use BRIN:
-- - Random value distribution (UUIDs, random amounts)
-- - Frequently updated columns
-- - Small tables (B-Tree is fine)
```

Q55. What is a covering index with INCLUDE?

Difficulty:
Medium

INCLUDE clause adds non-key columns to index leaf pages. Query can be satisfied without touching heap (Index Only Scan).

```
-- All needed columns in the index
CREATE INDEX idx_orders_cover ON orders(user_id)
  INCLUDE (amount, status, created_at);

-- This query = Index Only Scan (no heap access)
SELECT amount, status, created_at FROM orders WHERE user_id = 42;

-- vs composite index (old approach):
CREATE INDEX idx_orders_wide ON orders(user_id, amount, status, created_at);
-- INCLUDE is better: included columns not part of B-Tree key
-- → smaller index, no overhead in index traversal

EXPLAIN SELECT amount FROM orders WHERE user_id = 42;
-- "Index Only Scan" + "Heap Fetches: 0" = perfect
```

Q56. How do you find and fix slow queries?

Difficulty:
Medium

```
-- Step 1: Enable slow query log
ALTER SYSTEM SET log_min_duration_statement = 1000;
SELECT pg_reload_conf();
```

```
-- Step 2: pg_stat_statements (best tool)
SELECT query, calls, mean_exec_time, total_exec_time
FROM pg_stat_statements ORDER BY total_exec_time DESC LIMIT 10;

-- Step 3: EXPLAIN ANALYZE the slow query
EXPLAIN (ANALYZE, BUFFERS) SELECT ...;

-- Common fixes:
-- Seq Scan on large table → CREATE INDEX
-- "Rows Removed by Filter: 999000" → index isn't selective enough
-- "actual rows=10000 rows=10" → stale stats → ANALYZE
-- "Sort external merge Disk" → increase work_mem
-- N+1 queries → add JOIN or use IN/ANY
```

Q57. What is the leftmost prefix rule for composite indexes?

Difficulty:
Medium

```
CREATE INDEX idx_orders ON orders(user_id, status, created_at);

-- Uses full index:
WHERE user_id=1 AND status='paid' AND created_at>'2024-01-01'

-- Uses prefix (user_id + status):
WHERE user_id=1 AND status='paid'

-- Uses only leading column:
WHERE user_id=1

-- Does NOT use index (missing leading column):
WHERE status='paid' AND created_at>'2024-01-01'

-- Rule: put equality columns first, range column last
-- BAD: (created_at, user_id) for WHERE user_id=? AND created_at>?
-- GOOD: (user_id, created_at) for same query
```

Q58. What is Index Only Scan and when does it work?

Difficulty:
Medium

```
-- Requirements:
-- 1. All needed columns are in the index
-- 2. Pages marked as all-visible in visibility map (VACUUM ran)

CREATE INDEX idx_orders_cover ON orders(user_id) INCLUDE (amount);

EXPLAIN SELECT amount FROM orders WHERE user_id = 42;
-- "Index Only Scan using idx_orders_cover"
-- "Heap Fetches: 0" = visibility map used, no heap I/O

-- After bulk insert: run VACUUM to update visibility map
-- Otherwise: "Heap Fetches: N" = index only scan falls back to heap for unvacuumed pages
VACUUM orders;
```

Q59. How do you detect and handle index bloat?

Difficulty:
Medium

```
-- Check index sizes
SELECT indexrelname,
```

```

    pg_size_pretty(pg_relation_size(indexrelid)) AS size,
    idx_scan
FROM pg_stat_user_indexes
WHERE relname='orders'
ORDER BY pg_relation_size(indexrelid) DESC;

-- Index larger than table = bloat
SELECT i.relname AS index, c.relname AS table,
    pg_size_pretty(pg_relation_size(i.oid)) AS idx_size,
    pg_size_pretty(pg_relation_size(c.oid)) AS tbl_size
FROM pg_class c JOIN pg_index ix ON c.oid=ix.indrelid
JOIN pg_class i ON i.oid=ix.indexrelid
WHERE pg_relation_size(i.oid) > pg_relation_size(c.oid);

-- Fix: REINDEX CONCURRENTLY (no locks!)
REINDEX INDEX CONCURRENTLY idx_orders_user;
-- or pg_repack --index=idx_orders_user mydb

```

Q60. What is EXPLAIN FORMAT JSON useful for?

Difficulty:
Medium

```

-- JSON format for tooling (paste to https://explain.dalibo.com)
EXPLAIN (ANALYZE, BUFFERS, FORMAT JSON)
SELECT * FROM orders WHERE user_id=42;

-- Key metrics to extract:
-- Plan.Node Type, Actual Rows vs Plan Rows (estimate accuracy)
-- Shared Hit Blocks vs Read Blocks (cache hit rate)
-- Actual Total Time (per node timing)

-- Tip: paste output to explain.dalibo.com for visual plan
-- or pgMustard for automated recommendations

```

Q61. What is statistics target and extended statistics?

Difficulty: Hard

```

-- Default statistics_target = 100 (samples 300×100 rows)
-- Increase for columns with many distinct values used in JOINS/WHERE

ALTER TABLE orders ALTER COLUMN user_id SET STATISTICS 500;
ANALYZE orders;

-- Extended statistics for correlated columns
-- When WHERE state='CA' AND city='LA' has different selectivity than each alone
CREATE STATISTICS stat_state_city ON state, city FROM users;
ANALYZE users;

-- Verify
SELECT attname, n_distinct, most_common_vals, histogram_bounds
FROM pg_stats WHERE tablename='orders' AND attname='user_id';

```

Q62. What is the difference between IN and ANY?

Difficulty: Easy

```

-- Equivalent forms:
WHERE status IN ('a','b')
WHERE status = ANY(ARRAY['a','b'])

-- For parameterized arrays in Go (pass slice to ANY):

```

```
-- db.Query("SELECT * FROM users WHERE id = ANY($1)", pq.Array(ids))
-- pgx: db.Query(ctx, "SELECT * FROM users WHERE id = ANY($1)", ids)

-- NOT IN vs NOT EXISTS (NULL trap)
-- NOT IN: if subquery returns NULL → entire result empty
-- NOT EXISTS: safe, correct NULL handling

-- IN with subquery is often rewritten to semi-join by planner
```

Q63. What is parallel query?

Difficulty:
Medium

```
SET max_parallel_workers_per_gather = 4;
SET parallel_tuple_cost = 0.1;

-- Force parallel for testing
SET min_parallel_table_scan_size = 0;

EXPLAIN ANALYZE
SELECT date_trunc('day',created_at), SUM(amount)
FROM orders WHERE created_at > NOW()-INTERVAL '1 year'
GROUP BY 1;
-- "Gather" node with "Workers Planned: 4"
-- "Parallel Aggregate" under Gather

-- Disable if causing issues for small queries
SET max_parallel_workers_per_gather = 0;

-- Operations that parallelize:
-- Parallel Seq Scan, Index Scan, Hash Join, Aggregate
```

Q64. What is HOT update and fill factor?

Difficulty: Hard

```
-- HOT (Heap Only Tuple) update: row stays on same heap page
-- → no index update needed → much faster for non-indexed column updates

-- fillfactor: leave space for HOT updates
CREATE TABLE orders (...) WITH (fillfactor = 70); -- 30% free per page
CREATE INDEX idx_orders_status ON orders(status) WITH (fillfactor = 70);

-- Check HOT update ratio
SELECT relname,
       n_tup_hot_upd,
       n_tup_upd,
       round(n_tup_hot_upd::numeric/nullif(n_tup_upd,0)*100,1) AS hot_pct
FROM pg_stat_user_tables WHERE relname='orders';
-- HOT > 80% = good, most updates stay in same page
```

Q65. How do you use pg_hint_plan?

Difficulty: Hard

```
CREATE EXTENSION pg_hint_plan;

-- Force index scan
/*+ IndexScan(orders idx_orders_user) */
SELECT * FROM orders WHERE user_id = 42;

-- Force join order
```

```

/*+ Leading(u o) HashJoin(u o) */
SELECT u.name, COUNT(*) FROM users u JOIN orders o ON o.user_id=u.id GROUP BY u.name;

-- Disable parallel
/*+ NoParallel(orders) */
SELECT SUM(amount) FROM orders;

-- Available: SeqScan, IndexScan, BitmapScan, NestLoop, HashJoin, MergeJoin, Leading

```

Interview tip: "Use only as last resort. Fix statistics, refactor query, or tune cost params first. Hints become stale when data changes."

Q66. What is the N+1 query problem in PostgreSQL?

Difficulty:
Medium

```

-- N+1: 1 query for posts + N queries for each author
-- posts, _ := db.Query("SELECT id, user_id FROM posts LIMIT 100")
-- for each post: db.QueryRow("SELECT name FROM users WHERE id=?", post.UserID)
-- = 101 queries!

-- Fix 1: JOIN
SELECT p.*, u.name FROM posts p JOIN users u ON u.id=p.user_id LIMIT 100;

-- Fix 2: IN clause
SELECT id, name FROM users WHERE id = ANY($1::bigint[]);

-- Fix 3: unnest for batch fetch in Go
SELECT u.id, u.name FROM users u
JOIN unnest($1::bigint[]) AS ids(id) ON u.id = ids.id;

```

Q67. What is VACUUM ANALYZE vs ANALYZE?

Difficulty: Easy

```

ANALYZE orders;           -- collect statistics only (fast)
VACUUM ANALYZE orders;    -- reclaim dead tuples + collect stats

-- After bulk import: ALWAYS analyze
COPY orders FROM '/tmp/bulk.csv' CSV;
ANALYZE orders;           -- without this, planner uses stale stats → bad plans

-- Check last analyze times
SELECT relname, last_analyze, last_autoanalyze,
       last_vacuum, last_autovacuum, n_live_tup, n_dead_tup
FROM pg_stat_user_tables ORDER BY last_analyze NULLS FIRST;

```

Q68. What are prepared statements and their plan caching behavior?

Difficulty:
Medium

```

PREPARE get_orders (bigint) AS
  SELECT * FROM orders WHERE user_id=$1 ORDER BY created_at DESC LIMIT 20;

EXECUTE get_orders(42);
DEALLOCATE get_orders;

-- After 5 executions, PG switches to generic plan (parameter-agnostic)
-- Can be suboptimal for skewed data distributions
-- Force custom plan per-execution (safer for varying data)

```

```
SET plan_cache_mode = force_custom_plan;

-- Check prepared statements
SELECT name, statement, parameter_types FROM pg_prepared_statements;
```

Q69. What is the UNION vs UNION ALL performance difference?

Difficulty: Easy

```
-- UNION: deduplicates (requires sort or hash) → slower
SELECT user_id FROM orders_2023
UNION
SELECT user_id FROM orders_2024;

-- UNION ALL: keeps all rows, no dedup → faster
SELECT user_id FROM orders_2023
UNION ALL
SELECT user_id FROM orders_2024;

-- Rule: always use UNION ALL unless you need deduplication
-- If you need distinct: SELECT DISTINCT FROM (... UNION ALL ...) is clearer

-- INTERSECT (rows in both), EXCEPT (rows in first not second)
SELECT user_id FROM premium_users
INTERSECT SELECT user_id FROM active_orders;
```

Q70. How do you optimize GROUP BY queries?

Difficulty: Medium

```
-- Group on indexed column: much faster
CREATE INDEX idx_orders_status ON orders(status);
SELECT status, COUNT(*) FROM orders GROUP BY status;

-- Partial aggregate: filter first
SELECT status, COUNT(*) FROM orders
WHERE created_at > NOW()-INTERVAL '30 days' -- filter first (index)
GROUP BY status;

-- Grouping sets / rollup for multi-dimension
SELECT region, category, SUM(amount)
FROM sales
GROUP BY ROLLUP (region, category);
-- = GROUP BY region,category UNION GROUP BY region UNION GROUP BY ()

-- CUBE: all combinations
GROUP BY CUBE (region, category);
```

Q71. What is the query cost model and how do you tune work_mem?

Difficulty: Hard

```
-- work_mem: memory per sort/hash op
-- Per query, per operation → can be 5× work_mem for complex query

-- Detect when temp files created (= work_mem exceeded)
SELECT query, temp_blks_read, temp_blks_written
FROM pg_stat_statements WHERE temp_blks_written > 0
ORDER BY temp_blks_written DESC;

-- Check via EXPLAIN
```

```

EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM orders ORDER BY amount;
-- "Sort Method: external merge Disk: 1234kB" → increase work_mem
-- "Sort Method: quicksort Memory: 456kB" → fine

-- Per-session override for heavy reports
SET work_mem = '256MB';
-- ... run complex query ...
RESET work_mem;

-- log_temp_files = 10MB → log any temp file > 10MB

```

Q72. What is GROUPING SETS, ROLLUP, and CUBE?

Difficulty:
Medium

```

-- ROLLUP: hierarchical subtotals (year→month→day)
SELECT extract(year FROM sale_date) AS yr,
       extract(month FROM sale_date) AS mo,
       SUM(amount) AS rev
FROM sales
GROUP BY ROLLUP (extract(year FROM sale_date), extract(month FROM sale_date));

-- CUBE: all possible combinations
SELECT region, product_type, SUM(amount)
FROM sales GROUP BY CUBE (region, product_type);
-- = every combination: (region,product_type), (region), (product_type), ()

-- GROUPING SETS: explicit combinations
GROUP BY GROUPING SETS ((region,product_type),(region),(product_type),());

-- GROUPING() function: detect aggregate rows
SELECT region, GROUPING(region) AS is_subtotal FROM sales GROUP BY ROLLUP(region);

```

Q73. How do you do zero-downtime schema migrations?

Difficulty: Hard

```

-- Step 1: Add nullable column (instant, no table rewrite)
ALTER TABLE orders ADD COLUMN notes TEXT;

-- Step 2: Backfill in batches
DO $$ BEGIN LOOP
    UPDATE orders SET notes='' WHERE notes IS NULL
    AND id IN (SELECT id FROM orders WHERE notes IS NULL LIMIT 10000);
    EXIT WHEN NOT FOUND;
    PERFORM pg_sleep(0.1);
END LOOP; END; $$;

-- Step 3: Add NOT NULL constraint safely (PG12+)
ALTER TABLE orders ADD CONSTRAINT notes_not_null
CHECK (notes IS NOT NULL) NOT VALID;
ALTER TABLE orders VALIDATE CONSTRAINT notes_not_null; -- ShareUpdateExclusive (light)

-- Step 4: Drop old column (after old code retired)
ALTER TABLE orders DROP COLUMN old_column;

-- NEVER: ALTER TABLE ADD COLUMN NOT NULL DEFAULT now() on large table
-- (triggers full table rewrite in PG < 11, even PG11+ for variable defaults)

```


Q74. What is CREATE INDEX CONCURRENTLY?**Difficulty:**
Medium

```
-- Normal CREATE INDEX: requires SHARE lock (blocks writes)
-- CREATE INDEX CONCURRENTLY: only needs brief locks at start/end

CREATE INDEX CONCURRENTLY idx_orders_user ON orders(user_id);
-- Takes 3x longer but doesn't block INSERT/UPDATE/DELETE

-- Cannot run inside a transaction!
-- If cancelled: leaves INVALID index (must be dropped and recreated)

-- Check for invalid indexes
SELECT relname FROM pg_class WHERE relkind='i'
    AND NOT EXISTS (
        SELECT 1 FROM pg_index WHERE indexrelid=pg_class.oid AND indisvalid
    );

-- Drop invalid
DROP INDEX CONCURRENTLY idx_orders_user;
```

Q75. What is the difference between REINDEX and REINDEX CONCURRENTLY?**Difficulty:**
Medium

```
-- REINDEX: requires ACCESS EXCLUSIVE lock (blocks all queries)
REINDEX INDEX idx_orders_user;
REINDEX TABLE orders; -- rebuilds all indexes

-- REINDEX CONCURRENTLY (PG12+): builds replacement, then swaps
-- No blocking of reads or writes
REINDEX INDEX CONCURRENTLY idx_orders_user;
REINDEX TABLE CONCURRENTLY orders;

-- Use REINDEX CONCURRENTLY in production for index corruption or bloat fix
-- Check all invalid indexes after any failure
SELECT relname FROM pg_class WHERE relkind='i'
    AND NOT indisvalid FROM pg_index i WHERE i.indexrelid=pg_class.oid;
```

Q76. What is pgstattuple for bloat analysis?**Difficulty: Hard**

```
CREATE EXTENSION pgstattuple;

-- Table bloat
SELECT * FROM pgstattuple('orders');
-- table_len, tuple_count, tuple_len, dead_tuple_count, dead_tuple_len
-- free_space / table_len > 30% = bloated

-- Index bloat
SELECT * FROM pgstatindex('idx_orders_user');
-- avg_leaf_density < 70% = index bloated

-- Per-page analysis
SELECT * FROM pgstattuple_approx('orders'); -- faster, approximate
```

Q77. What are the ANALYZE and VACUUM parameters for bulk load optimization?**Difficulty:**
Medium

```
-- For bulk imports: disable autovacuum temporarily
ALTER TABLE staging SET (autovacuum_enabled = off);

-- Bulk load
COPY staging FROM '/tmp/large_import.csv' CSV HEADER;
-- or with parallel workers:
-- Use multiple COPY statements in parallel sessions

-- Re-enable and run manually
ALTER TABLE staging SET (autovacuum_enabled = on);
VACUUM ANALYZE staging;

-- After COPY: statistics are stale, planner will use wrong estimates
-- ANALYZE is mandatory after large data changes

-- Disable indexes during load (rebuild after)
-- DROP INDEX idx_staging_col; ... COPY ... CREATE INDEX CONCURRENTLY;
```

Q78. What is the difference between index scan and index only scan performance?**Difficulty:**
Medium

```
-- Index Scan: B-tree lookup → random heap fetch for each row
-- 1000 matching rows → 1000 random I/Os to heap (slow on HDD, OK on SSD)

-- Index Only Scan: B-tree lookup → data from index leaf page
-- No heap access → dramatically faster

-- Preconditions for Index Only Scan:
-- 1. All queried columns must be in index (including INCLUDE columns)
-- 2. Visibility map must mark pages as all-visible (VACUUM needed)

-- Force visibility map update
VACUUM orders;
EXPLAIN SELECT amount FROM orders WHERE user_id=42;
-- Should show: "Index Only Scan" with "Heap Fetches: 0"
```

Q79. What is a multicolumn index vs multiple single-column indexes?**Difficulty:**
Medium

```
-- Multicolumn (a, b): one index, serves queries on (a) and (a,b)
CREATE INDEX idx_user_status ON orders(user_id, status);

-- Two separate indexes: planner can combine with Bitmap Index Scan
CREATE INDEX idx_user ON orders(user_id);
CREATE INDEX idx_status ON orders(status);

-- When to prefer multicolumn:
-- High-frequency query on (a AND b) together
-- Need covering index (INCLUDE)

-- When to prefer separate:
-- Each column queried independently often
-- Table is heavily written (fewer indexes = faster writes)
```

```
-- Planner can merge two separate indexes using Bitmap Index Scan
-- EXPLAIN shows "BitmapAnd" node
```

Q80. How do you handle NULL values in indexes?

Difficulty:
Medium

```
-- B-Tree indexes include NULLs (unlike some other databases)
-- NULL IS NULL query uses index

-- IS NULL query uses index
SELECT * FROM orders WHERE shipped_at IS NULL;
-- Will use index on shipped_at if it exists

-- Partial index for faster IS NULL queries
CREATE INDEX idx_orders_unshipped ON orders(id)
  WHERE shipped_at IS NULL; -- only indexes unshipped orders

-- Null position in sort (NULLS FIRST / NULLS LAST)
SELECT * FROM orders ORDER BY shipped_at NULLS LAST;
CREATE INDEX idx_orders_shipped ON orders(shipped_at NULLS LAST);
```

Transactions & Concurrency

Q81. What are PostgreSQL's isolation levels?

Difficulty: Hard

```
-- Read Committed (default): sees only committed data; re-reads may differ
-- Repeatable Read: snapshot at txn start; prevents non-repeatable reads + phantoms (PG-specific!)
-- Serializable (SSI): full isolation; conflicts detected, one txn rolled back

BEGIN TRANSACTION ISOLATION LEVEL REPEATABLE READ;
SELECT balance FROM accounts WHERE id=1; -- sees 1000
-- Another session commits UPDATE accounts SET balance=2000 WHERE id=1
SELECT balance FROM accounts WHERE id=1; -- still 1000 (snapshot)
COMMIT;

-- Serializable example: prevents write skew
BEGIN ISOLATION LEVEL SERIALIZABLE;
SELECT COUNT(*) FROM doctors WHERE on_call=true; -- 2, both go off-call
UPDATE doctors SET on_call=false WHERE id=1;
COMMIT;
-- One transaction succeeds, other gets:
-- ERROR: could not serialize access due to read/write dependencies

-- PostgreSQL's REPEATABLE READ also prevents phantoms (unlike standard SQL)
```

Q82. What is a deadlock and how do you prevent it?

Difficulty:
Medium

```
-- Deadlock: T1 holds A wants B; T2 holds B wants A → circular wait
-- PostgreSQL detects via lock dependency graph; kills one victim

-- Prevention: always acquire locks in consistent order
-- Sort IDs before updating: min(id) first
```

```
BEGIN;
UPDATE accounts SET balance=balance-100 WHERE id=LEAST($1,$2);
UPDATE accounts SET balance=balance+100 WHERE id=GREATEST($1,$2);
COMMIT;

-- Check current deadlock history in log:
-- LOG: deadlock detected
-- Set deadlock_timeout = '1s' (default) – check after 1s wait

-- Check locks
SELECT * FROM pg_locks WHERE NOT granted;
```

Q83. What is SELECT FOR UPDATE and SKIP LOCKED?

Difficulty:
Medium

```
-- SELECT FOR UPDATE: row-level exclusive lock
BEGIN;
SELECT * FROM jobs WHERE status='pending'
ORDER BY created_at LIMIT 1
FOR UPDATE SKIP LOCKED; -- skip already-locked rows
UPDATE jobs SET status='running', started_at=NOW() WHERE id=$1;
COMMIT;

-- SKIP LOCKED: multiple workers pick different jobs without blocking
-- Without SKIP LOCKED: all workers queue behind first locked row

-- Variants:
FOR UPDATE           -- exclusive, wait for lock
FOR UPDATE NOWAIT    -- fail immediately if locked
FOR UPDATE SKIP LOCKED -- skip locked rows (best for queues)
FOR SHARE            -- shared lock, allows other FOR SHARE
FOR KEY SHARE        -- weakest, only blocks FOR UPDATE
```

Q84. What is optimistic vs pessimistic locking?

Difficulty:
Medium

```
-- Pessimistic: lock before read
BEGIN;
SELECT * FROM inventory WHERE id=42 FOR UPDATE; -- blocks others
UPDATE inventory SET qty=qty-1 WHERE id=42 AND qty>0;
COMMIT;

-- Optimistic: check version on update
-- Read:
SELECT id, qty, version FROM inventory WHERE id=42;
-- Returns: {id:42, qty:10, version:5}

-- Update (check version):
UPDATE inventory SET qty=qty-1, version=version+1
WHERE id=42 AND version=5; -- 0 rows = conflict → retry
GET DIAGNOSTICS updated = ROW_COUNT;
IF updated = 0 THEN RAISE EXCEPTION 'conflict, retry'; END IF;

-- Optimistic: better for read-heavy, rare conflicts
-- Pessimistic: better for write-heavy, frequent conflicts
```

Q85. What is the outbox pattern with PostgreSQL?**Difficulty: Hard**

```
-- Problem: write to DB + publish to Kafka = two systems, not atomic
-- Solution: write to outbox table in same transaction

CREATE TABLE outbox (
  id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  event_type TEXT NOT NULL,
  payload JSONB NOT NULL,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  processed_at TIMESTAMPTZ
);

BEGIN;
INSERT INTO orders (user_id, amount) VALUES (42, 99.99) RETURNING id;
INSERT INTO outbox (event_type, payload)
  VALUES ('order.created', '{"order_id":1,"user_id":42}');
COMMIT; -- atomic! both or neither

-- Outbox poller (Go goroutine):
rows := db.Query("SELECT id,event_type,payload FROM outbox
  WHERE processed_at IS NULL ORDER BY id LIMIT 100 FOR UPDATE SKIP LOCKED")
-- publish to Kafka
-- UPDATE outbox SET processed_at=NOW() WHERE id=ANY($1)

-- Or: Debezium CDC reads outbox via WAL (near-real-time, no polling)
```

Q86. What is advisory locking?**Difficulty: Medium**

```
-- Application-managed locks using arbitrary integers
SELECT pg_try_advisory_lock(42);      -- try, returns true/false
SELECT pg_advisory_lock(42);         -- wait until acquired
SELECT pg_advisory_unlock(42);       -- release

-- Transaction-level (auto-released on COMMIT/ROLLBACK)
SELECT pg_try_advisory_xact_lock(42); -- no explicit unlock

-- Distributed cron: ensure only one pod runs a job
IF pg_try_advisory_lock(hashtext('daily_report')) THEN
  BEGIN
    -- do work
  EXCEPTION WHEN OTHERS THEN
    PERFORM pg_advisory_unlock(hashtext('daily_report')); RAISE;
  END;
  PERFORM pg_advisory_unlock(hashtext('daily_report'));
END IF;

-- Distributed lock table as alternative
INSERT INTO distributed_locks (key, holder, expires_at)
VALUES ('job', 'worker-01', NOW()+INTERVAL '5 min')
ON CONFLICT (key) DO UPDATE SET holder=EXCLUDED.holder, expires_at=EXCLUDED.expires_at
WHERE distributed_locks.expires_at < NOW();
```

Q87. What are PostgreSQL lock modes?**Difficulty: Hard**

```
-- Lock hierarchy (blocks all modes to its right):
-- ACCESS SHARE (SELECT) → ROW SHARE (SELECT FOR UPDATE) →
-- ROW EXCLUSIVE (INSERT/UPDATE/DELETE) →
-- SHARE UPDATE EXCLUSIVE (VACUUM/CREATE INDEX CONCURRENTLY) →
-- SHARE (CREATE INDEX) → SHARE ROW EXCLUSIVE →
-- EXCLUSIVE → ACCESS EXCLUSIVE (ALTER TABLE/TRUNCATE/DROP)

-- Most dangerous: ALTER TABLE = ACCESS EXCLUSIVE (blocks everything)
-- Use lock_timeout to prevent indefinite blocking:
SET lock_timeout = '5s';
ALTER TABLE orders ADD COLUMN notes TEXT;
-- ERROR if can't acquire in 5s

-- Safe DDL pattern:
SET lock_timeout = '2s';
LOOP: try ALTER TABLE; if timeout, sleep 100ms, retry
```

Q88. What is long transaction danger?

Difficulty:
Medium

```
-- Long transactions:
-- 1. Hold row locks → block other queries
-- 2. Prevent VACUUM (can't reclaim rows visible to old snapshot)
-- 3. Cause table bloat
-- 4. Increase crash recovery time

-- Detect long/idle-in-transaction sessions
SELECT pid, now()-xact_start AS duration, state, query
FROM pg_stat_activity
WHERE state='idle in transaction' AND now()-state_change > INTERVAL '1 min';

-- Prevention
idle_in_transaction_session_timeout = '5min' -- auto-kill
statement_timeout = '30s' -- kill runaway queries

-- Kill offenders
SELECT pg_terminate_backend(pid) FROM pg_stat_activity
WHERE state='idle in transaction' AND now()-state_change > INTERVAL '10 min';
```

Q89. What are savepoints?

Difficulty: Easy

```
BEGIN;
INSERT INTO orders (user_id, amount) VALUES (42, 100);
SAVEPOINT after_order;

INSERT INTO order_items (order_id, product_id, qty) VALUES (1, 99, 2);
-- Out of stock! Roll back only the item insert
ROLLBACK TO SAVEPOINT after_order;
INSERT INTO order_items (order_id, product_id, qty) VALUES (1, 100, 2); -- substitute
RELEASE SAVEPOINT after_order;
COMMIT; -- order + substitute item committed

-- In Go with pgx:
tx.Exec(ctx, "SAVEPOINT sp1")
_, err = tx.Exec(ctx, "INSERT ...")
if err != nil {
    tx.Exec(ctx, "ROLLBACK TO SAVEPOINT sp1")
}
```

```
}
```

Q90. What is SERIALIZABLE isolation and SSI?

Difficulty: Hard

```
-- PostgreSQL uses Serializable Snapshot Isolation (SSI):
-- NOT traditional locking-based 2PL
-- Tracks read/write dependencies → detects anomalies → rolls back one txn

-- Classic write-skew example (NOT caught by Repeatable Read):
-- Two doctors both check "2 on-call" → both go off-call → 0 doctors!

BEGIN ISOLATION LEVEL SERIALIZABLE;
SELECT COUNT(*) FROM oncall WHERE active=true; -- 2
UPDATE oncall SET active=false WHERE doctor_id=1;
COMMIT;
-- One succeeds; other gets: ERROR: could not serialize access

-- Retry pattern for serializable failures
FOR attempt 1..3:
    BEGIN ISOLATION LEVEL SERIALIZABLE
    ... do work ...
    COMMIT → if 40001 (serialization_failure): retry
    if 40P01 (deadlock_detected): retry

-- SSI overhead: minimal for low-conflict workloads
-- High-conflict: more retries → more latency
```

Q91. What is two-phase commit (2PC)?

Difficulty: Hard

```
-- Prepare: transaction is ready to commit but not yet committed
BEGIN;
UPDATE accounts SET balance=balance-100 WHERE id=1;
PREPARE TRANSACTION 'txn-transfer-001';

-- Transaction is now durable, suspended
-- Check prepared transactions
SELECT * FROM pg_prepared_xacts;

-- Commit (in same or different session)
COMMIT PREPARED 'txn-transfer-001';
-- Or rollback
ROLLBACK PREPARED 'txn-transfer-001';

-- WARNING: orphaned prepared transactions prevent VACUUM
-- Monitor: SELECT * FROM pg_prepared_xacts WHERE prepared < NOW()-INTERVAL '1h'
-- In practice: Saga + Outbox pattern preferred over 2PC for microservices
```

Q92. What is the difference between ROW EXCLUSIVE and FOR UPDATE?

Difficulty: Hard

```
-- ROW EXCLUSIVE: table-level lock acquired by INSERT/UPDATE/DELETE
-- FOR UPDATE: row-level lock within the transaction

-- Example:
BEGIN;
UPDATE orders SET status='shipped' WHERE id=42;
-- Acquires: ROW EXCLUSIVE on orders table (blocks ALTER TABLE)
```

```
-- + row-level lock on row id=42 (blocks other FOR UPDATE on same row)

-- FOR UPDATE vs FOR NO KEY UPDATE:
-- FOR UPDATE: blocks inserts of FK references to this row
-- FOR NO KEY UPDATE: allows FK-referencing inserts (weaker, less contention)
-- Regular UPDATE automatically uses FOR NO KEY UPDATE semantics

-- FOR KEY SHARE: weakest row lock, only blocks FOR UPDATE
-- Used internally for FK checks
```

Q93. What is deferred constraint checking vs immediate?

Difficulty:
Medium

```
-- INITIALLY IMMEDIATE: constraint checked after each statement (default)
-- INITIALLY DEFERRED: constraint checked at COMMIT

CREATE TABLE nodes (
  id INT PRIMARY KEY,
  next_id INT REFERENCES nodes(id) DEFERRABLE INITIALLY DEFERRED
);

-- Can insert circular references in same transaction
BEGIN;
INSERT INTO nodes VALUES (1, 2); -- references 2 (doesn't exist yet)
INSERT INTO nodes VALUES (2, 1); -- references 1 (now exists)
COMMIT; -- both referenced nodes exist → OK

-- Override per-transaction
SET CONSTRAINTS ALL DEFERRED;
SET CONSTRAINTS fk_user IMMEDIATE;
```

Q94. What is pg_locks and how do you diagnose contention?

Difficulty:
Medium

```
-- Full lock view
SELECT a.pid, a.query, a.state, l.relation::regclass, l.mode, l.granted
FROM pg_locks l JOIN pg_stat_activity a ON l.pid=a.pid
WHERE l.relation IS NOT NULL
ORDER BY l.relation, l.mode;

-- pg_blocking_pids() – simplest way to find blockers
SELECT pid, pg_blocking_pids(pid) AS blockers, query
FROM pg_stat_activity
WHERE cardinality(pg_blocking_pids(pid)) > 0;

-- Lock types: relation, transactionid, tuple, page, object
-- Most common: relation (table-level) and transactionid (row-level)
```

Q95. What is phantom read and which isolation level prevents it?

Difficulty: Hard

```
-- Phantom read: re-read of range returns different rows in same transaction

-- Read Committed: phantoms possible
BEGIN;
SELECT COUNT(*) FROM orders WHERE user_id=42; -- 5
-- Another session inserts a new order for user 42 and commits
```



```

SELECT COUNT(*) FROM orders WHERE user_id=42; -- 6 (phantom!)
COMMIT;

-- Repeatable Read: PostgreSQL prevents phantoms (unlike SQL standard!)
BEGIN ISOLATION LEVEL REPEATABLE READ;
SELECT COUNT(*) FROM orders WHERE user_id=42; -- 5
-- Another session inserts + commits
SELECT COUNT(*) FROM orders WHERE user_id=42; -- still 5 (snapshot)
COMMIT;

-- Standard SQL: only SERIALIZABLE prevents phantoms
-- PostgreSQL: REPEATABLE READ also prevents phantoms (stronger than standard)

```

Replication & High Availability

Q96. How does streaming replication work?

Difficulty: Hard

```

-- Primary streams WAL records to standby via TCP
-- Standby applies WAL → maintains read-only copy

-- postgresql.conf on primary:
-- wal_level = replica
-- max_wal_senders = 10

-- Create replication user
CREATE ROLE replicator WITH REPLICATION LOGIN PASSWORD 'pass';

-- On standby: pg_basebackup
pg_basebackup -h primary -U replicator -D /pgdata -Fp -Xs -P -R
-- -R: creates standby.signal + postgresql.auto.conf with primary_conninfo

-- Monitor replication
SELECT client_addr, state, sent_lsn, replay_lsn,
       pg_wal_lsn_diff(sent_lsn, replay_lsn) AS lag_bytes,
       now()-reply_time AS last_heartbeat
FROM pg_stat_replication;

-- On standby:
SELECT now()-pg_last_xact_replay_timestamp() AS lag;

```

Q97. What is synchronous vs asynchronous replication?

Difficulty:
Medium

```

-- Async (default): primary commits without waiting for standby
-- Zero write overhead, small risk of data loss on primary crash

-- Synchronous: primary waits for standby confirmation
synchronous_standby_names = 'standby1' -- wait for this standby
synchronous_commit = on                -- wait for WAL flush on standby

-- Modes:
synchronous_commit = remote_write      -- standby wrote to OS buffer (fast + safe)
synchronous_commit = remote_apply      -- standby applied changes (reads consistent)

```

```
synchronous_commit = on          -- WAL flushed on standby to disk

-- Per-transaction override
BEGIN;
SET LOCAL synchronous_commit = off; -- async for this write (non-critical)
INSERT INTO analytics (...);
COMMIT;
```

Q98. What is logical replication?

Difficulty: Hard

```
-- Physical: byte-for-byte WAL copy, same version required, whole cluster
-- Logical: row-level changes, can filter tables, replicate to different versions

-- wal_level = logical (required)

-- Publisher
CREATE PUBLICATION my_pub FOR TABLE orders, users;
CREATE PUBLICATION all_tables FOR ALL TABLES;

-- Subscriber (can be different PG version!)
CREATE SUBSCRIPTION my_sub
  CONNECTION 'host=primary dbname=myapp user=replicator'
  PUBLICATION my_pub;

-- Monitor
SELECT * FROM pg_stat_subscription;

-- Use cases:
-- Zero-downtime major version upgrade (PG14 → PG16)
-- Selective replication (subset of tables)
-- Multi-datacenter fanout
-- Real-time analytics replica
```

Q99. What is Patroni?

Difficulty: Medium

```
# Patroni: HA template for PostgreSQL using etcd/ZooKeeper/Consul
# Leader election + automatic failover

scope: postgres-cluster
etcd: {host: etcd:2379}

bootstrap:
  dcs:
    ttl: 30
    loop_wait: 10
    retry_timeout: 30
    maximum_lag_on_failover: 1048576 # 1MB max lag

postgresql:
  listen: 0.0.0.0:5432
  parameters:
    wal_level: replica
    hot_standby: on
    max_wal_senders: 5

patronictl list          # cluster status
```

```
patronictl switchover          # manual failover
patronictl failover           # force failover
curl :8008/health              # is this node leader?
# RTO < 30s for automatic failover
```

Q100. What is pg_basebackup and WAL archiving?

Difficulty:
Medium

```
# Full backup
pg_basebackup -h localhost -D /backup/base -Ft -z -Xs -P --checkpoint=fast

# WAL archiving for PITR
# postgresql.conf:
archive_mode = on
archive_command = 'aws s3 cp %p s3://backups/wal/%f'

# Restore to point in time:
# restore_command = 'aws s3 cp s3://backups/wal/%f %p'
# recovery_target_time = '2024-01-15 14:30:00 UTC'
# recovery_target_action = 'promote'
# touch recovery.signal

# Verify backup works!
# Test restore monthly. Untested backups = no backups.
```

Q101. ### Q101–Q120: Replication Deep Dives

Difficulty:
Medium

Q102. What is a replication slot?

Difficulty: Hard

```
-- Logical replication slot
SELECT pg_create_logical_replication_slot('debezium', 'pgoutput');

-- Physical replication slot
SELECT pg_create_physical_replication_slot('standby1');

-- CRITICAL: unused slots retain WAL indefinitely → disk full!
SELECT slot_name, active,
       pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(), confirmed_flush_lsn)) AS lag
FROM pg_replication_slots;

-- Limit WAL retention per slot
ALTER SYSTEM SET max_slot_wal_keep_size = '5GB';

-- Drop unused slot
SELECT pg_drop_replication_slot('old_slot');
```

Q103. What is CDC (Change Data Capture) with Debezium?

Difficulty: Hard

```
// Debezium PostgreSQL connector config
{
  "connector.class": "io.debezium.connector.postgresql.PostgresConnector",
  "database.hostname": "postgres",
  "database.port": "5432",
  "database.user": "debezium",
```

```

    "database.dbname": "myapp",
    "database.server.name": "myapp",
    "slot.name": "debezium",
    "plugin.name": "pgoutput",
    "publication.name": "cdc_pub",
    "table.include.list": "public.orders,public.users"
}

-- Create publication for CDC
CREATE PUBLICATION cdc_pub FOR ALL TABLES;

-- CDC output: before/after images of every change → Kafka
-- Use cases: cache invalidation, Elasticsearch sync, audit logs

```

Q104. What is hot standby and how do you route reads to it?

Difficulty:
Medium

```

-- hot_standby = on (postgresql.conf on standby)
-- hot_standby_feedback = on (tell primary about standby queries)
-- max_standby_streaming_delay = 30s

-- On standby
SELECT pg_is_in_recovery(); -- true
SELECT now()-pg_last_xact_replay_timestamp() AS lag;

-- Application routing in Go:
primaryDB := connectTo("primary:5432")
standbyDB := connectTo("standby:5432")

// Non-critical reads from standby (may be seconds stale)
func getProducts(ctx context.Context) { query(standbyDB) }
// Writes and read-after-write to primary
func createOrder(ctx context.Context) { query(primaryDB) }

```

Q105. How do you monitor replication lag?

Difficulty:
Medium

```

-- On primary: bytes of WAL not yet replayed
SELECT client_addr,
       pg_size_pretty(pg_wal_lsn_diff(sent_lsn, replay_lsn)) AS lag_size,
       extract(epoch FROM now()-reply_time) AS seconds_since_heartbeat
FROM pg_stat_replication;

-- On standby: time lag
SELECT now()-pg_last_xact_replay_timestamp() AS replay_lag;

-- Alerts:
-- lag_bytes > 100MB → standby falling behind
-- replay_lag > 30s → investigate standby load or network
-- seconds_since_heartbeat > 60 → standby may be down

-- Prometheus metrics via postgres_exporter:
-- pg_replication_slots_lag_bytes
-- pg_stat_replication_lag_seconds

```

Q106. What is pg_upgrade for major version upgrades?

Difficulty:
Medium

```
# pg_upgrade: in-place upgrade (minutes downtime)
systemctl stop postgresql-14
pg_upgrade \
  -b /usr/pgsql-14/bin -B /usr/pgsql-16/bin \
  -d /var/lib/pgsql/14/data -D /var/lib/pgsql/16/data \
  --check # dry run first

# --link: hard links (instant, but old cluster unusable)
# --clone: copy-on-write on btrfs/ZFS (nearly instant)

# After upgrade:
/usr/pgsql-16/bin/vacuumdb --all --analyze-in-stages

# Zero-downtime: logical replication PG14→PG16
# Set up subscription, wait for lag→0, flip connection string
```

Q107. What is Patroni failover and how does it work?

Difficulty:
Medium

Patroni components:

- etcd/ZooKeeper/Consul: distributed consensus store
- Patroni process: runs on each PostgreSQL node

Failover process:

1. Primary fails to renew TTL lease in etcd (TTL=30s)
2. Most up-to-date standby acquires leader key in etcd
3. New leader promotes (pg_promote() or promote.signal)
4. Other standbys reconfigure to follow new primary
5. HAProxy health checks → routes traffic to new primary

RT0: 30-60 seconds (TTL expiry + promotion + DNS/LB update)

RPO: 0 if synchronous_commit=on; seconds if async

Q108. What is pg_stat_replication and what should you alert on?

Difficulty: Easy

```
SELECT application_name, client_addr, state,
       sent_lsn, write_lsn, flush_lsn, replay_lsn,
       pg_wal_lsn_diff(sent_lsn, replay_lsn) AS total_lag,
       sync_state -- async, sync, potential, quorum
FROM pg_stat_replication;
```

-- Alert conditions:

- 1. Row count drops (standby disconnected)
- 2. total_lag > 100MB (standby falling behind)
- 3. now()-reply_time > 60s (no heartbeat from standby)
- 4. state != 'streaming' (replication not flowing)

-- Grafana: pg_stat_replication.{sent_lsn - replay_lsn} metric

Q109. What is cascade replication?

Difficulty:
Medium

- Primary → Standby1 → Standby2 (cascade)
- Reduces load on primary (doesn't send WAL to all standbys directly)
- On Standby2: point to Standby1 as upstream

```
-- primary_conninfo = 'host=standby1 ...'

-- recovery_min_apply_delay = 4h -- intentional delay for protection
-- Delayed standby: 4-hour window to recover from accidental mass delete

-- recovery_target_time = '...' -- used for PITR from delayed standby
-- hot_standby = on -- still accepts read queries
```

Q110. What is a synchronous_standby_names quorum?

Difficulty: Hard

```
-- ANY 1 (standby1, standby2, standby3)
-- = commit waits for ANY 1 of 3 standbys
synchronous_standby_names = 'ANY 1 (standby1,standby2,standby3)'

-- FIRST 1 (standby1, standby2)
-- = commit waits for standby1 (priority), falls back to standby2
synchronous_standby_names = 'FIRST 1 (standby1,standby2)'

-- ALL standbys (no quorum syntax, all must acknowledge)
synchronous_standby_names = 'standby1,standby2'
-- Only use for extreme durability; any standby failure = primary stalls

-- ANY 1 of 3 = tolerates 1 standby failure, strong durability
-- Recommended for production HA
```

Q111. What is promote and demotion in PostgreSQL HA?

Difficulty: Medium

```
-- Promote standby to primary
-- Method 1: pg_promote() function (PostgreSQL 12+)
SELECT pg_promote();

-- Method 2: promote signal file
touch /var/lib/postgresql/data/promote.signal
-- PostgreSQL detects file, ends recovery, becomes primary

-- Method 3: pg_ctl promote
pg_ctl promote -D /var/lib/postgresql/data

-- After promotion: configure as new primary
-- Old primary (if it recovers): must be reconfigured as standby
-- Not automatic: needs manual intervention or Patroni

-- pg_rewind: resync old primary to new timeline
pg_rewind -D /var/lib/postgresql/data --source-server="host=new-primary..."
```

Q112. ### Q111-Q120: Additional HA/Replication

Difficulty: Medium

Q	Topic
Q111	WAL archiving to S3 with recovery configuration
Q112	max_standby_streaming_delay and recovery conflicts
Q113	Logical decoding with pgoutput plugin

Q114	Replication identity: FULL vs DEFAULT vs NOTHING
Q115	pg_receivewal for WAL streaming backup
Q116	Hot standby performance: parallel workers on replicas
Q117	Two-datacenter HA: Active-Active vs Active-Passive
Q118	pg_failover_slots extension for logical slots on standby
Q119	Read replica connection pooling with PgBouncer
Q120	RTO and RPO definitions and achieving targets

Partitioning & Sharding

Q113. What is declarative table partitioning?

Difficulty: Hard

```
-- RANGE partitioning (most common: by date)
CREATE TABLE orders (
  id BIGINT NOT NULL,
  user_id BIGINT NOT NULL,
  amount NUMERIC(10,2),
  created_at TIMESTAMPTZ NOT NULL
) PARTITION BY RANGE (created_at);

CREATE TABLE orders_2024_01 PARTITION OF orders
  FOR VALUES FROM ('2024-01-01') TO ('2024-02-01');
CREATE TABLE orders_2024_02 PARTITION OF orders
  FOR VALUES FROM ('2024-02-01') TO ('2024-03-01');
CREATE TABLE orders_default PARTITION OF orders DEFAULT;

-- Index on parent → created on all partitions
CREATE INDEX idx_orders_user ON orders(user_id, created_at);

-- Partition pruning (query only hits relevant partitions)
EXPLAIN SELECT * FROM orders WHERE created_at BETWEEN '2024-01-01' AND '2024-02-01';
-- "Partitions removed: 11 out of 12"
```

Q114. What is LIST and HASH partitioning?

Difficulty: Medium

```
-- LIST: categorical data
CREATE TABLE sales (region TEXT, amount NUMERIC)
PARTITION BY LIST (region);
CREATE TABLE sales_us PARTITION OF sales FOR VALUES IN ('US','CA','MX');
CREATE TABLE sales_eu PARTITION OF sales FOR VALUES IN ('UK','DE','FR');
CREATE TABLE sales_other PARTITION OF sales DEFAULT;

-- HASH: even distribution (no natural range/list key)
CREATE TABLE users (id BIGINT, email TEXT)
PARTITION BY HASH (id);
CREATE TABLE users_p0 PARTITION OF users FOR VALUES WITH (MODULUS 4, REMAINDER 0);
```

```
CREATE TABLE users_p1 PARTITION OF users FOR VALUES WITH (MODULUS 4, REMAINDER 1);
-- ... p2, p3

-- HASH limitation: range queries scan ALL partitions
-- Use HASH only for write distribution, not range queries
```

Q115. How do you manage partitions (add/remove)?

Difficulty:
Medium

```
-- Add new partition (non-blocking)
CREATE TABLE orders_2024_04 PARTITION OF orders
  FOR VALUES FROM ('2024-04-01') TO ('2024-05-01');

-- Detach partition (PG14+ non-blocking)
ALTER TABLE orders DETACH PARTITION orders_2023_01;
-- Now it's a standalone table

-- Drop partition (INSTANT – drops entire file)
DROP TABLE orders_2023_01;
-- 100-1000x faster than DELETE + VACUUM on same data

-- Attach existing table
ALTER TABLE orders ATTACH PARTITION orders_2024_05
  FOR VALUES FROM ('2024-05-01') TO ('2024-06-01');

-- List partitions
SELECT inhrelid::regclass AS partition,
  pg_get_expr(relpartbound, oid) AS bounds
FROM pg_inherits JOIN pg_class ON oid=inhrelid
WHERE inhparent='orders'::regclass;
```

Q116. What is partition pruning?

Difficulty:
Medium

```
-- Automatic: planner eliminates partitions WHERE clause can't match

-- PRUNING WORKS:
EXPLAIN SELECT * FROM orders WHERE created_at = '2024-01-15';
-- Only scans orders_2024_01

-- PRUNING FAILS (filter not on partition key):
EXPLAIN SELECT * FROM orders WHERE user_id = 42;
-- Scans ALL partitions (need index on user_id in each partition)

-- Runtime pruning (PG12+): works with parameterized queries
PREPARE p(timestamptz) AS SELECT * FROM orders WHERE created_at = $1;
EXPLAIN EXECUTE p('2024-01-15'); -- prunes at runtime

SET enable_partition_pruning = on; -- default on
```

Q117. What is pg_partman?

Difficulty:
Medium

```
CREATE EXTENSION pg_partman;

SELECT partman.create_parent(
  p_parent_table => 'public.events',
```



```

p_control => 'created_at',
p_type => 'range',
p_interval => 'monthly',
p_premake => 3 -- create 3 future partitions ahead
);

UPDATE partman.part_config SET
    retention = '12 months',
    retention_keep_table = false,
    infinite_time_partitions = true
WHERE parent_table = 'public.events';

SELECT partman.run_maintenance();

-- Schedule with pg_cron
SELECT cron.schedule('partman-maintenance','0 1 * * *',
    'SELECT partman.run_maintenance()');
```

Q118. What are sharding strategies for PostgreSQL?

Difficulty: Hard

Options for horizontal scaling beyond single PostgreSQL:

1. Application-level sharding
 - App routes to different PostgreSQL instances by shard key
 - `hash(user_id) % N` → shard DB
 - Simple but cross-shard queries require app joins
 - No automatic rebalancing
2. Citus (extension)
 - Distributes PostgreSQL tables across worker nodes
 - `SELECT * FROM orders WHERE user_id=42` → routed to correct shard
 - Supports: distributed tables, reference tables, co-located joins
 - Coordinator node + worker nodes
3. Foreign Data Wrappers (postgres_fdw)
 - Local table + remote tables on different servers
 - Can JOIN across servers (coordinator does merge)
 - Pushes predicates to remote servers
4. PgBouncer + multiple databases
 - Connection-level routing to different DBs
 - `database = myapp_{user}` → per-tenant database

Q119. What is Citus and distributed PostgreSQL?

Difficulty: Hard

```

-- Citus turns PostgreSQL into distributed DBMS
CREATE EXTENSION citus;

-- Distribute table by shard key
SELECT create_distributed_table('orders', 'user_id');
-- Shards orders across worker nodes by hash(user_id)

-- Co-locate related tables (same shard key = same node)
SELECT create_distributed_table('order_items', 'user_id',
    colocate_with => 'orders');
```

-- Reference table: replicated to all workers

```
SELECT create_reference_table('products');

-- Distributed query (coordinator routes automatically)
SELECT user_id, SUM(amount)
FROM orders WHERE created_at > '2024-01-01'
GROUP BY user_id;
-- Coordinator sends subqueries to relevant workers, merges results
```

Q120. What is partition-wise join and aggregate?**Difficulty: Hard**

```
-- Partition-wise join: join matching partitions directly
-- (instead of joining whole tables)
SET enable_partitionwise_join = on;
SET enable_partitionwise_aggregate = on;

-- If orders and order_items both partitioned by user_id:
SELECT o.user_id, COUNT(i.id)
FROM orders o JOIN order_items i ON i.order_id = o.id
GROUP BY o.user_id;

-- With partition-wise join:
-- orders_p0 JOIN order_items_p0 → aggregate
-- orders_p1 JOIN order_items_p1 → aggregate
-- Final merge (parallel)
-- Much faster than cross-partition join

-- Requires same partition key and matching partition bounds
```

Q121. What is sub-partitioning?**Difficulty: Hard**

```
-- Partition by date, then by region within each date
CREATE TABLE events (
    id BIGINT,
    region TEXT,
    created_at TIMESTAMPTZ
) PARTITION BY RANGE (created_at);

CREATE TABLE events_2024_01 PARTITION OF events
    FOR VALUES FROM ('2024-01-01') TO ('2024-02-01')
    PARTITION BY LIST (region); -- sub-partition by region

CREATE TABLE events_2024_01_us PARTITION OF events_2024_01
    FOR VALUES IN ('US', 'CA');
CREATE TABLE events_2024_01_eu PARTITION OF events_2024_01
    FOR VALUES IN ('UK', 'DE', 'FR');

-- Query prunes both levels:
SELECT * FROM events
WHERE created_at BETWEEN '2024-01-01' AND '2024-02-01'
    AND region = 'US';
-- Scans only events_2024_01_us
```

Q122. What are the partitioning limitations in PostgreSQL?**Difficulty:
Medium**

```
-- Limitations:
```

```
-- 1. Global unique indexes: must include partition key
-- CREATE UNIQUE INDEX ON orders(id, user_id) -- must include user_id
-- CREATE UNIQUE INDEX ON orders(id) -- ERROR: unique indexes on partitioned tables must include all par

-- 2. Foreign keys to partitioned table: must reference partition key
-- 3. Triggers: can't create row-level triggers on partitioned table (only on partitions)
-- 4. BEFORE triggers don't fire on parent, only on partitions

-- Workarounds:
-- Unique across partitions: application-level dedup or Citus
-- FK to partitioned: use application-level enforcement
-- Triggers on parent: STATEMENT-level only

-- What works well:
-- Range queries on partition key (with pruning)
-- Partition DROP for retention (instant)
-- Different storage per partition
-- Parallel scans across partitions
```

Advanced Features & Operations

Q123. What are PostgreSQL extensions you must know?

Difficulty: Easy

```
CREATE EXTENSION pg_stat_statements; -- query performance
CREATE EXTENSION pgcrypto;           -- gen_random_uuid(), encrypt
CREATE EXTENSION pg_trgm;             -- LIKE/ILIKE indexes
CREATE EXTENSION uuid-ossdp;          -- uuid_generate_v4()
CREATE EXTENSION hstore;              -- key-value column
CREATE EXTENSION ltree;               -- hierarchical data paths
CREATE EXTENSION intarray;            -- array operations
CREATE EXTENSION postgres_fdw;        -- foreign data wrapper
CREATE EXTENSION pg_partman;          -- partition management
CREATE EXTENSION pg_cron;             -- scheduled jobs
CREATE EXTENSION pgaudit;             -- audit logging
CREATE EXTENSION btree_gist;          -- exclusion constraints on scalars
SELECT * FROM pg_available_extensions ORDER BY name;
```

Q124. What is full-text search in PostgreSQL?

Difficulty: Hard

```
-- Generated FTS column
ALTER TABLE articles ADD COLUMN fts TSVECTOR
    GENERATED ALWAYS AS (
        setweight(to_tsvector('english', coalesce(title, '')), 'A') ||
        setweight(to_tsvector('english', coalesce(body, '')), 'B')
    ) STORED;

CREATE INDEX idx_articles_fts ON articles USING GIN(fts);

-- Search with ranking and highlight
SELECT id, title,
    ts_rank(fts, q) AS rank,
    ts_headline('english', body, q, 'MaxFragments=2') AS excerpt
FROM articles, to_tsquery('english', 'postgres & (index|performance)') q
```

```
WHERE fts @@ q
ORDER BY rank DESC LIMIT 20;

-- Phrase search
WHERE fts @@ phraseto_tsquery('english','query planner')
```

Q125. What is pg_cron?**Difficulty: Easy**

```
CREATE EXTENSION pg_cron;

SELECT cron.schedule('refresh-views','*/15 * * * *',
  'REFRESH MATERIALIZED VIEW CONCURRENTLY daily_revenue');

SELECT cron.schedule('cleanup','0 2 * * *',
  'DELETE FROM sessions WHERE expires_at < NOW()');

SELECT cron.schedule('partman','0 1 * * *',
  'SELECT partman.run_maintenance()');

-- View schedule
SELECT jobid, schedule, command, active FROM cron.job;

-- View history
SELECT jobid, status, start_time, return_message
FROM cron.job_run_details ORDER BY start_time DESC LIMIT 20;

SELECT cron.unschedule('refresh-views');
```

Q126. What is NOTIFY/LISTEN?**Difficulty: Medium**

```
-- Trigger that sends notification on change
CREATE OR REPLACE FUNCTION notify_order_change() RETURNS TRIGGER AS $$
BEGIN
  PERFORM pg_notify('order_changes',
    json_build_object('op',TG_OP,'id',COALESCE(NEW.id,OLD.id))::text);
  RETURN COALESCE(NEW, OLD);
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER orders_notify AFTER INSERT OR UPDATE OR DELETE ON orders
  FOR EACH ROW EXECUTE FUNCTION notify_order_change();

-- Subscriber (manual)
LISTEN order_changes;

-- In Go with pgx (dedicated connection required!):
conn.Exec(ctx, "LISTEN order_changes")
n, _ := conn.WaitForNotification(ctx)
// n.Channel, n.Payload
```

Q127. What is RLS and how do you implement multi-tenancy?**Difficulty: Hard**

```
-- Multi-tenant schema: every table has tenant_id
ALTER TABLE orders ENABLE ROW LEVEL SECURITY;
ALTER TABLE orders FORCE ROW LEVEL SECURITY;
```

```
CREATE POLICY tenant_policy ON orders
  FOR ALL TO app_role
  USING (tenant_id = current_setting('app.tenant_id')::bigint)
  WITH CHECK (tenant_id = current_setting('app.tenant_id')::bigint);

-- In Go: set context on each transaction
tx, _ := db.BeginTx(ctx, nil)
tx.Exec("SET LOCAL app.tenant_id = $1", tenantID)
// all queries auto-filtered by tenant

-- PgBouncer + transaction pooling limitation:
-- SET doesn't persist across pooled connections
-- Use SET LOCAL inside transaction or session pooling
```

Q128. What are stored procedures vs functions?

Difficulty:
Medium

```
-- Function: returns value, cannot COMMIT/ROLLBACK
CREATE OR REPLACE FUNCTION get_user_total(p_user_id BIGINT)
RETURNS NUMERIC AS $$
  SELECT COALESCE(SUM(amount),0)
  FROM orders WHERE user_id=p_user_id AND status='paid';
$$ LANGUAGE sql STABLE; -- STABLE = same input → same output within txn

-- Procedure (PG11+): can manage transactions
CREATE OR REPLACE PROCEDURE process_batch(p_size INT)
LANGUAGE plpgsql AS $$
DECLARE processed INT;
BEGIN
  LOOP
    UPDATE jobs SET status='done'
    WHERE id IN (SELECT id FROM jobs WHERE status='pending' LIMIT p_size);
    GET DIAGNOSTICS processed = ROW_COUNT;
    COMMIT; -- commit each batch
    EXIT WHEN processed < p_size;
  END LOOP;
END;
$$;
CALL process_batch(1000);

-- Volatility: VOLATILE | STABLE | IMMUTABLE
-- IMMUTABLE: can be used in expression indexes
```

Q129. What is the exclusion constraint?

Difficulty: Hard

```
CREATE EXTENSION btree_gist; -- for scalar + range exclusions

-- No two overlapping reservations for same room
CREATE TABLE bookings (
  room_id INT,
  during TSTZRANGE
);

ALTER TABLE bookings ADD CONSTRAINT no_overlap
  EXCLUDE USING GIST (
    room_id WITH =, -- same room
    during WITH && -- overlapping period
```

```
);

INSERT INTO bookings VALUES (1, '[2024-01-10 9:00, 2024-01-10 10:00)');
INSERT INTO bookings VALUES (1, '[2024-01-10 9:30, 2024-01-10 11:00)');
-- ERROR: conflicting key value violates exclusion constraint

-- Also: prevent overlapping price ranges, IP ranges, etc.
```

Q130. What is FDW (Foreign Data Wrapper)?

Difficulty:
Medium

```
CREATE EXTENSION postgres_fdw;

CREATE SERVER remote_db
  FOREIGN DATA WRAPPER postgres_fdw
  OPTIONS (host 'remote-host', port '5432', dbname 'analytics');

CREATE USER MAPPING FOR current_user SERVER remote_db
  OPTIONS (user 'ruser', password 'pass');

IMPORT FOREIGN SCHEMA public FROM SERVER remote_db INTO remote_schema;

-- Query remote table like local
SELECT * FROM remote_schema.reports WHERE date > '2024-01-01';

-- Join local + remote
SELECT o.id, r.summary FROM orders o
JOIN remote_schema.reports r ON r.order_id = o.id;

-- Pushes WHERE clauses to remote server when possible
-- Use for: migrations, cross-DB analytics, legacy system access
```

Q131. What is pg_stat_user_tables for health monitoring?

Difficulty: Easy

```
SELECT relname,
       seq_scan, idx_scan,
       n_tup_ins, n_tup_upd, n_tup_del,
       n_tup_hot_upd,
       n_live_tup, n_dead_tup,
       round(n_dead_tup::numeric/nullif(n_live_tup,0)*100,1) AS dead_pct,
       last_vacuum, last_autovacuum, last_analyze
FROM pg_stat_user_tables
ORDER BY n_dead_tup DESC;

-- Red flags:
-- seq_scan >> idx_scan on large table = missing index
-- dead_pct > 20% = VACUUM needed urgently
-- last_autovacuum IS NULL or very old = autovacuum not running
-- n_tup_hot_upd/n_tup_upd < 50% = consider lower fillfactor
```

Q132. What is pg_activity and real-time monitoring?

Difficulty: Easy

```
# pg_activity: top-like for PostgreSQL
pip install pg_activity
pg_activity -U postgres -d mydb

# Key columns:
```

```
# CPU, MEM, READ, WRITE, TIME, W (wait event), state, query

# pgBadger: analyze slow query logs
pgbadger /var/log/postgresql/pg14.log -o report.html

# prometheus postgres_exporter: key metrics
# pg_up, pg_stat_activity_count, pg_stat_database_blks_hit_ratio
# pg_stat_user_tables_n_dead_tup, pg_replication_slots_lag_bytes
```

Q133. ### Q141-Q145: Operations**Difficulty: Medium****Q134. What is VACUUM FULL vs pg_repack?****Difficulty: Medium**

```
-- VACUUM FULL: rewrites entire table, reclaims disk space to OS
-- Requires ACCESS EXCLUSIVE lock (blocks ALL queries for duration)
VACUUM FULL orders; -- never in production during business hours!

-- pg_repack: online table rebuild
-- 1. Creates new table
-- 2. Applies changes via triggers during rebuild
-- 3. Brief ACCESS EXCLUSIVE lock at final swap only
pg_repack --table=orders mydb -- minutes, not hours of blocking

-- When to use each:
-- Maintenance window available + table is huge: VACUUM FULL
-- Production, can't block reads/writes: pg_repack
-- Bloat < 30%: VACUUM ANALYZE (usually enough)
```

Q135. What is idle_in_transaction_session_timeout?**Difficulty: Easy**

```
-- Set globally (postgresql.conf)
idle_in_transaction_session_timeout = '5min'

-- Or per-role
ALTER ROLE app_user SET idle_in_transaction_session_timeout = '5min';

-- What it does: terminates connections that are
-- in "idle in transaction" state for more than N milliseconds
-- "idle in transaction" = BEGIN; ... (no activity) ... (holding locks)

-- This is the most impactful single setting for preventing
-- accidental lock pile-ups in production

-- Also set:
statement_timeout = '30s' -- kill runaway queries
lock_timeout = '10s' -- fail fast on lock waits
```

Q136. How do you handle large table alterations?**Difficulty: Hard**

```
-- ADD COLUMN with DEFAULT (PG11+: constant default = no rewrite)
ALTER TABLE orders ADD COLUMN priority INT DEFAULT 0; -- instant in PG11+
-- Variable default still rewrites:
ALTER TABLE orders ADD COLUMN updated_at TIMESTAMPTZ DEFAULT NOW(); -- slow!
```

```
-- Better: add nullable, backfill, add constraint
ALTER TABLE orders ADD COLUMN updated_at TIMESTAMPTZ; -- instant
UPDATE orders SET updated_at=created_at WHERE updated_at IS NULL; -- batch
ALTER TABLE orders ALTER COLUMN updated_at SET DEFAULT NOW(); -- instant

-- Column rename (PG10+: instant)
ALTER TABLE orders RENAME COLUMN old_name TO new_name;

-- Type change (requires table rewrite):
-- Add new column → backfill → swap → drop old
ALTER TABLE orders ADD COLUMN amount_v2 BIGINT;
UPDATE orders SET amount_v2 = (amount * 100)::BIGINT; -- pennies
ALTER TABLE orders ALTER COLUMN amount_v2 SET NOT NULL;
-- rename + drop old
```

Q137. What is pg_cron and how do you use it safely?

Difficulty:
Medium

```
-- pg_cron only runs on primary (not on standbys)
-- For cluster-wide jobs: advisory lock ensures single execution

CREATE OR REPLACE FUNCTION run_if_primary()
RETURNS VOID AS $$
BEGIN
    IF pg_is_in_recovery() THEN RETURN; END IF; -- skip on standby
    IF NOT pg_try_advisory_lock(hashtext('my_job')) THEN RETURN; END IF;
    BEGIN
        -- do work
        DELETE FROM old_sessions WHERE expires_at < NOW();
        PERFORM pg_advisory_unlock(hashtext('my_job'));
    EXCEPTION WHEN OTHERS THEN
        PERFORM pg_advisory_unlock(hashtext('my_job')); RAISE;
    END;
END;
$$ LANGUAGE plpgsql;

SELECT cron.schedule('cleanup','0 * * * *','SELECT run_if_primary());
```

Q138. What is logical decoding and how does it enable CDC?

Difficulty: **Hard**

```
-- wal_level = logical (required)
SELECT pg_create_logical_replication_slot('cdc_slot','pgoutput');

-- Read raw WAL changes
SELECT * FROM pg_logical_slot_get_changes('cdc_slot', NULL, NULL,
    'proto_version','1', 'publication_names','my_pub');

-- For Debezium: reads via pgoutput, sends to Kafka
-- Key field: "before" + "after" images of every row change

-- Monitor slot lag (CRITICAL: unused slot fills disk)
SELECT slot_name, active,
    pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(), confirmed_flush_lsn)) AS lag
FROM pg_replication_slots;

-- Protection:
ALTER SYSTEM SET max_slot_wal_keep_size = '10GB'; -- limit WAL retention
```


Go + PostgreSQL Patterns

Q139. What is the best PostgreSQL driver for Go?

Difficulty: Easy

```
// pgx v5: recommended (PostgreSQL-specific, fast, full-featured)
import (
    "github.com/jackc/pgx/v5"
    "github.com/jackc/pgx/v5/pgxpool"
    "github.com/jackc/pgx/v5/stdlib" // database/sql adapter
)

// pgxpool: production connection pool
config, _ := pgxpool.ParseConfig(os.Getenv("DATABASE_URL"))
config.MaxConns = 25
config.MinConns = 5
config.MaxConnIdleTime = 5 * time.Minute
config.MaxConnLifetime = 30 * time.Minute

pool, _ := pgxpool.NewWithConfig(ctx, config)

// database/sql adapter if needed
db := stdlib.OpenDBFromPool(pool)
db.SetMaxOpenConns(25)
db.SetMaxIdleConns(5)
```

Q140. How do you handle transactions in Go?

Difficulty: Easy

```
func (r *Repo) CreateOrder(ctx context.Context, o *Order) error {
    tx, err := r.db.BeginTx(ctx, &sql.TxOptions{
        Isolation: sql.LevelReadCommitted,
    })
    if err != nil { return fmt.Errorf("begin: %w", err) }
    defer tx.Rollback() // no-op after Commit()

    var orderID int64
    err = tx.QueryRowContext(ctx,
        "INSERT INTO orders (user_id,amount) VALUES ($1,$2) RETURNING id",
        o.UserID, o.Amount,
    ).Scan(&orderID)
    if err != nil { return fmt.Errorf("insert order: %w", err) }

    for _, item := range o.Items {
        _, err = tx.ExecContext(ctx,
            "INSERT INTO order_items (order_id,product_id,qty) VALUES ($1,$2,$3)",
            orderID, item.ProductID, item.Qty)
        if err != nil { return fmt.Errorf("insert item: %w", err) }
    }
    return tx.Commit()
}
```

Q141. How do you handle PostgreSQL errors in Go?**Difficulty:**
Medium

```
import "github.com/jackc/pgx/v5/pgconn"

var (
    ErrDuplicate      = errors.New("duplicate record")
    ErrNotFound       = errors.New("not found")
    ErrForeignKey     = errors.New("referenced record missing")
    ErrConflict       = errors.New("concurrent modification, retry")
)

func mapPgError(err error) error {
    var pgErr *pgconn.PgError
    if !errors.As(err, &pgErr) { return err }

    switch pgErr.Code {
    case "23505": return fmt.Errorf("%w: %s", ErrDuplicate, pgErr.Detail)
    case "23503": return fmt.Errorf("%w: %s", ErrForeignKey, pgErr.Detail)
    case "40001": return ErrConflict // serialization_failure → retry
    case "40P01": return ErrConflict // deadlock_detected → retry
    case "57014": return context.DeadlineExceeded // query canceled
    default:     return fmt.Errorf("db error %s: %s", pgErr.Code, pgErr.Message)
    }
}

// Retry pattern for serializable failures
func withRetry(ctx context.Context, fn func() error) error {
    for i := 0; i < 3; i++ {
        err := fn()
        if err == nil { return nil }
        if !errors.Is(err, ErrConflict) { return err }
        time.Sleep(time.Duration(i*50) * time.Millisecond)
    }
    return errors.New("conflict after 3 retries")
}
```

Q142. How do you implement bulk insert with COPY in Go?**Difficulty:** **Hard**

```
import "github.com/jackc/pgx/v5"

func BulkInsert(ctx context.Context, pool *pgxpool.Pool, users []User) error {
    conn, err := pool.Acquire(ctx)
    if err != nil { return err }
    defer conn.Release()

    rows := make([][]interface{}, len(users))
    for i, u := range users {
        rows[i] = []interface{}{u.Name, u.Email, u.CreatedAt}
    }

    count, err := conn.Conn().CopyFrom(
        ctx,
        pgx.Identifier{"users"},
        []string{"name", "email", "created_at"},
        pgx.CopyFromRows(rows),
    )
}
```

```

    log.Printf("inserted %d rows via COPY", count)
    return err
}

// Alternative: unnest for moderate volumes
func BatchInsert(ctx context.Context, db *pgxpool.Pool, users []User) error {
    names := make([]string, len(users))
    emails := make([]string, len(users))
    for i, u := range users { names[i]=u.Name; emails[i]=u.Email }

    _, err := db.Exec(ctx, `
        INSERT INTO users (name, email)
        SELECT * FROM unnest($1::text[], $2::text[])`,
        names, emails)
    return err
}

```

Q143. How do you implement the repository pattern with Go and PostgreSQL?

Difficulty:
Medium

```

type UserRepository interface {
    GetByID(ctx context.Context, id int64) (*User, error)
    GetByEmail(ctx context.Context, email string) (*User, error)
    Create(ctx context.Context, user *User) error
    List(ctx context.Context, page, size int) ([]*User, int64, error)
}

type postgresUserRepo struct{ pool *pgxpool.Pool }

func (r *postgresUserRepo) GetByID(ctx context.Context, id int64) (*User, error) {
    var u User
    err := r.pool.QueryRow(ctx,
        "SELECT id,name,email,created_at FROM users WHERE id=$1 AND deleted_at IS NULL",
        id,
    ).Scan(&u.ID, &u.Name, &u.Email, &u.CreatedAt)

    if errors.Is(err, pgx.ErrNoRows) { return nil, ErrNotFound }
    if err != nil { return nil, mapPgError(err) }
    return &u, nil
}

func (r *postgresUserRepo) Create(ctx context.Context, u *User) error {
    return r.pool.QueryRow(ctx,
        "INSERT INTO users(name,email) VALUES($1,$2) RETURNING id,created_at",
        u.Name, u.Email,
    ).Scan(&u.ID, &u.CreatedAt)
}

```

Q144. How do you implement cursor-based pagination in Go?

Difficulty:
Medium

```

type Cursor struct{ CreatedAt time.Time; ID int64 }

func (r *Repo) ListOrders(ctx context.Context, userID int64, cursor *Cursor, limit int) ([]*Order, *Cursor,
    var rows pgx.Rows
    var err error
    fetch := limit + 1

```

```

    if cursor == nil {
        rows, err = r.db.Query(ctx,
            `SELECT id,amount,created_at FROM orders
            WHERE user_id=$1 ORDER BY created_at DESC, id DESC LIMIT $2`,
            userID, fetch)
    } else {
        rows, err = r.db.Query(ctx,
            `SELECT id,amount,created_at FROM orders
            WHERE user_id=$1 AND (created_at,id) < ($2,$3)
            ORDER BY created_at DESC, id DESC LIMIT $4`,
            userID, cursor.CreatedAt, cursor.ID, fetch)
    }
    if err != nil { return nil, nil, err }
    defer rows.Close()

    orders := make([]*Order, 0, limit)
    for rows.Next() {
        var o Order
        rows.Scan(&o.ID, &o.Amount, &o.CreatedAt)
        orders = append(orders, &o)
    }

    var nextCursor *Cursor
    if len(orders) > limit {
        orders = orders[:limit]
        last := orders[len(orders)-1]
        nextCursor = &Cursor{CreatedAt: last.CreatedAt, ID: last.ID}
    }
    return orders, nextCursor, nil
}

```

Q145. How do you run database migrations in Go?

Difficulty:
Medium

```

import (
    "github.com/golang-migrate/migrate/v4"
    _ "github.com/golang-migrate/migrate/v4/database/postgres"
    _ "github.com/golang-migrate/migrate/v4/source/file"
)

func RunMigrations(databaseURL string) error {
    m, err := migrate.New("file://./migrations", databaseURL)
    if err != nil { return fmt.Errorf("migrate init: %w", err) }
    defer m.Close()

    if err = m.Up(); err != nil && !errors.Is(err, migrate.ErrNoChange) {
        return fmt.Errorf("migrate up: %w", err)
    }
    v, _, _ := m.Version()
    log.Printf("migrations applied, current version: %d", v)
    return nil
}

// migrations/000001_create_users.up.sql
// migrations/000001_create_users.down.sql
// migrations/000002_add_orders.up.sql
// Run at startup (before accepting requests)

```

```
// Use advisory lock to prevent concurrent migrations on multi-pod deploy
```

Q146. How do you use pgx batching?

Difficulty: Hard

```
func GetMultipleUsers(ctx context.Context, conn *pgx.Conn, ids []int64) ([]*User, error) {
    batch := &pgx.Batch{}
    for _, id := range ids {
        batch.Queue("SELECT id,name,email FROM users WHERE id=$1", id)
    }

    br := conn.SendBatch(ctx, batch)
    defer br.Close()

    var users []*User
    for range ids {
        var u User
        err := br.QueryRow().Scan(&u.ID, &u.Name, &u.Email)
        if err != nil {
            if errors.Is(err, pgx.ErrNoRows) { continue }
            return nil, err
        }
        users = append(users, &u)
    }
    return users, nil
}
// All N queries sent in 1 round-trip (vs N round-trips without batching)
// Especially beneficial across network (high latency connections)
```

Q147. How do you implement LISTEN/NOTIFY in Go?

Difficulty: Hard

```
func StartListener(ctx context.Context, dsn string, handler func(payload string)) {
    for {
        if err := listen(ctx, dsn, handler); err != nil {
            if errors.Is(err, context.Canceled) { return }
            log.Printf("listener error: %v, reconnecting in 5s", err)
            select {
                case <-ctx.Done(): return
                case <-time.After(5 * time.Second):
            }
        }
    }
}

func listen(ctx context.Context, dsn string, handler func(string)) error {
    // MUST use dedicated connection (not pool)
    conn, err := pgx.Connect(ctx, dsn)
    if err != nil { return err }
    defer conn.Close(ctx)

    if _, err = conn.Exec(ctx, "LISTEN order_changes"); err != nil {
        return err
    }

    for {
        n, err := conn.WaitForNotification(ctx)
        if err != nil { return err }
    }
}
```

```

        handler(n.Payload)
    }
}

```

Q148. How do you use pgx with OpenTelemetry tracing?

Difficulty: Hard

```

import (
    "github.com/jackc/pgx/v5"
    oteltrace "go.opentelemetry.io/otel/trace"
)

// Implement pgx.QueryTracer interface
type otelTracer struct{ tracer oteltrace.Tracer }

func (t *otelTracer) TraceQueryStart(ctx context.Context, _ *pgx.Conn, data pgx.TraceQueryStartData) context.Context {
    ctx, span := t.tracer.Start(ctx, "db.query")
    span.SetAttributes(attribute.String("db.statement", data.SQL))
    return ctx
}

func (t *otelTracer) TraceQueryEnd(ctx context.Context, _ *pgx.Conn, data pgx.TraceQueryEndData) {
    span := trace.SpanFromContext(ctx)
    if data.Err != nil { span.RecordError(data.Err) }
    span.End()
}

// Register tracer
config.ConnConfig.Tracer = &otelTracer{tracer: otel.Tracer("db")}

```

Q149. What is sqlc and how do you use it?

Difficulty: Medium

```

-- queries.sql
-- name: GetUser :one
SELECT id, name, email, created_at FROM users WHERE id = $1 AND deleted_at IS NULL;

-- name: CreateUser :one
INSERT INTO users (name, email) VALUES ($1, $2) RETURNING id, created_at;

-- name: ListOrdersByUser :many
SELECT id, amount, status, created_at FROM orders
WHERE user_id = $1 ORDER BY created_at DESC LIMIT $2;

# sqlc.yaml
version: "2"
sql:
  - engine: "postgresql"
    queries: "queries.sql"
    schema: "schema.sql"
    gen:
      go:
        package: "db"
        out: "db/"

// Generated type-safe code:
q := db.New(pool)
user, err := q.GetUser(ctx, 42) // returns *User, nil
orders, err := q.ListOrdersByUser(ctx, db.ListOrdersByUserParams{UserID: 42, Limit: 20})

```

Q150. How do you handle JSONB in Go?**Difficulty:**
Medium

```
import "encoding/json"

type EventPayload struct {
    EventType string    `json:"event_type"`
    UserID    int64    `json:"user_id"`
    Meta      map[string]any `json:"meta,omitempty"`
}

// Store JSONB
payload := EventPayload{EventType: "click", UserID: 42}
data, _ := json.Marshal(payload)
db.Exec(ctx,
    "INSERT INTO events (payload) VALUES ($1)",
    data) // pgx accepts []byte as jsonb

// Read JSONB
var raw []byte
db.QueryRow(ctx, "SELECT payload FROM events WHERE id=$1", id).Scan(&raw)
var p EventPayload
json.Unmarshal(raw, &p)

// pgx native: scan into struct with pgtype.JSON
var p EventPayload
db.QueryRow(ctx, "SELECT payload FROM events WHERE id=$1", id).Scan(&p)
// pgx v5 scans jsonb directly into struct via json.Unmarshal
```

Q151. How do you implement connection retry on startup in Go?**Difficulty:** Easy

```
func connectWithRetry(ctx context.Context, dsn string, maxAttempts int) (*pgxpool.Pool, error) {
    config, err := pgxpool.ParseConfig(dsn)
    if err != nil { return nil, err }

    backoff := time.Second
    for attempt := 1; attempt <= maxAttempts; attempt++ {
        pool, err := pgxpool.NewWithConfig(ctx, config)
        if err == nil {
            if pingErr := pool.Ping(ctx); pingErr == nil {
                log.Printf("connected to DB on attempt %d", attempt)
                return pool, nil
            }
            pool.Close()
        }
        if attempt == maxAttempts { return nil, fmt.Errorf("failed after %d attempts", maxAttempts) }
        log.Printf("DB not ready (attempt %d/%d), retrying in %v", attempt, maxAttempts, backoff)
        select {
        case <-ctx.Done(): return nil, ctx.Err()
        case <-time.After(backoff):
            backoff = min(backoff*2, 30*time.Second)
        }
    }
    return nil, errors.New("unreachable")
}
```

Q152. How do you write testable database code in Go?**Difficulty:**
Medium

```
// Option 1: Interface-driven (mock in tests)
type UserStore interface {
    GetUser(ctx context.Context, id int64) (*User, error)
    CreateUser(ctx context.Context, u *User) error
}

// Service depends on interface, not concrete DB
type UserService struct{ store UserStore }

// In tests: mock
type mockUserStore struct{ users map[int64]*User }
func (m *mockUserStore) GetUser(ctx context.Context, id int64) (*User, error) {
    if u, ok := m.users[id]; ok { return u, nil }
    return nil, ErrNotFound
}

// Option 2: testcontainers (real PostgreSQL in tests)
import "github.com/testcontainers/testcontainers-go"
container, _ := postgres.RunContainer(ctx,
    testcontainers.WithImage("postgres:16"),
    postgres.WithDatabase("testdb"),
    postgres.WithPassword("test"),
)
connStr, _ := container.ConnectionString(ctx)
// run migrations, test with real DB
defer container.Terminate(ctx)
```

Q153. How do you use pgx with custom types?**Difficulty:** **Hard**

```
import "github.com/jackc/pgx/v5/pgtype"

// Register custom PostgreSQL enum type
typeMap := pgtype.NewMap()
typeMap.RegisterDefaultPgType(OrderStatus(""), "order_status")

// Custom scanner for domain types
type Money struct{ Amount int64; Currency string }

func (m *Money) ScanRow(row pgx.Row) error {
    return row.Scan(&m.Amount, &m.Currency)
}

// pgtype built-in support
var t pgtype.Timestamptz
db.QueryRow(ctx, "SELECT created_at FROM orders WHERE id=$1", 1).Scan(&t)
goTime := t.Time // time.Time

// UUID
var id pgtype.UUID
db.QueryRow(ctx, "SELECT id FROM users WHERE email=$1", email).Scan(&id)
// id.Bytes = [16]byte
```


Q154. ### Q161-Q185: Advanced Topics**Difficulty:**
Medium

Q	Topic
Q161	pg_stat_io (PG16): per-object I/O stats
Q162	effective_cache_size vs shared_buffers difference
Q163	wal_compression for reducing WAL size
Q164	pg_prewarm: preload tables into shared buffers
Q165	logical_decoding_work_mem tuning
Q166	max_worker_processes for background workers
Q167	pgaudit: compliance audit logging extension
Q168	pg_walinspect: inspect WAL contents directly
Q169	Implementing outbox with Debezium CDC pipeline
Q170	Connection string parameters: options, search_path
Q171	Monitoring pg_stat_database for database-level metrics
Q172	enable_hashjoin, enable_seqscan: planner knobs
Q173	pg_config command: build configuration
Q174	Recovery: restore_command and recovery_target options
Q175	Parallel CREATE INDEX and max_parallel_maintenance_workers
Q176	SSL/TLS configuration for PostgreSQL connections
Q177	pg_hba.conf: host-based authentication
Q178	COPY security: COPY TO/FROM FILE vs COPY stdin/stdout
Q179	Monitoring bloat with pgstattuple
Q180	pg_stat_wal: WAL generation statistics
Q181	default_statistics_target global vs per-column
Q182	log_lock_waits for detecting lock contention
Q183	PostgreSQL wire protocol: extended vs simple query
Q184	pgx v5 pipeline mode for reduced round trips
Q185	Kubernetes readiness probe for PostgreSQL in Go

Q155. ### Q186-Q200: Go Integration Deep Dives**Difficulty:**
Medium

Q	Topic
Q186	pgx RowToStruct scanning pattern

Q187	db.Stats() for connection pool health monitoring
Q188	Context cancellation propagation through pgx queries
Q189	Implementing distributed lock with pg advisory locks in Go
Q190	Health check endpoint with PostgreSQL ping in Go HTTP server
Q191	pgx QueryExecMode: SimpleProtocol vs CachedPlan for PgBouncer
Q192	Handling PostgreSQL arrays with pgx ([]int64, []string)
Q193	pgx CopyFrom with custom CopyFromSource for streaming
Q194	Implementing pagination with keyset in REST API (Go)
Q195	Using sqlc for type-safe queries: setup and workflow
Q196	pgx Tracer interface for custom query logging
Q197	Testing with pgxmock vs testcontainers: trade-offs
Q198	Golang migrate: advisory lock for safe concurrent migrations
Q199	pgx v5 named arguments and arg structs
Q200	Implementing CDC consumer in Go with pglogrepl

Master these 200 questions and you'll handle any PostgreSQL interview at SDE2 level. Key areas: MVCC, indexing (B-Tree, GIN, partial), transactions (isolation levels, deadlocks, FOR UPDATE SKIP LOCKED), replication (streaming, logical, Patroni), and Go patterns (pgx, batch operations, error handling). ■

Additional PostgreSQL Questions (Q156–Q200)

Q156. What is pg_partman for partition management?

Difficulty: Hard

```
-- pg_partman: automates partition creation and maintenance
-- Supports: time-based and id-based range partitioning

-- Install extension
CREATE EXTENSION pg_partman SCHEMA partman;

-- Create partitioned table
CREATE TABLE orders (
    id BIGSERIAL,
    created_at TIMESTAMPTZ NOT NULL,
    customer_id BIGINT,
    amount DECIMAL(12,2),
    status TEXT
) PARTITION BY RANGE (created_at);
```

```

-- Setup partman management
SELECT partman.create_parent(
    p_parent_table := 'public.orders',
    p_control := 'created_at',
    p_interval := '1 month',    -- monthly partitions
    p_start_partition := '2024-01-01'
);

-- Partman will automatically:
-- Create future partitions (premake = 4 months ahead)
-- Archive/drop old partitions based on retention policy

-- Configure retention
UPDATE partman.part_config SET
    retention = '12 months',      -- keep 12 months
    retention_keep_table = false, -- drop old partitions
    infinite_time_partitions = true, -- keep creating new ones
    premake = 4                  -- create 4 future partitions
WHERE parent_table = 'public.orders';

-- Run maintenance (schedule this with pg_cron)
SELECT partman.run_maintenance_proc();

-- Or: pg_cron job
SELECT cron.schedule('partition-maintenance', '0 * * * *',
    'SELECT partman.run_maintenance_proc();');

```

Q157. What is PostgreSQL LISTEN/NOTIFY?

Difficulty: Hard

```

// Real-time notifications without polling

// PostgreSQL side:
// NOTIFY channel_name, 'optional payload';
// LISTEN channel_name;

// Trigger-based notification (most common)
CREATE OR REPLACE FUNCTION notify_order_change() RETURNS trigger AS $$
BEGIN
    PERFORM pg_notify('order_updates',
        json_build_object(
            'id', NEW.id,
            'status', NEW.status,
            'customer_id', NEW.customer_id
        )::text
    );
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER order_change_notify
    AFTER INSERT OR UPDATE ON orders
    FOR EACH ROW EXECUTE FUNCTION notify_order_change();

// Go listener (pgx v5)
conn, _ := pgx.Connect(ctx, dsn)

if _, err := conn.Exec(ctx, "LISTEN order_updates"); err != nil {

```

```

    panic(err)
}

for {
    notification, err := conn.WaitForNotification(ctx)
    if err != nil {
        if errors.Is(err, context.Canceled) { return }
        log.Printf("notification error: %v", err)
        // Reconnect...
        continue
    }

    var payload map[string]interface{}
    json.Unmarshal([]byte(notification.Payload), &payload)
    processOrderUpdate(payload)
}

// NOTIFY limitations:
// Payload max 8000 bytes
// If connection drops, notifications lost (not durable)
// Use for: real-time updates, cache invalidation, dev tooling
// Don't use for: durable event streaming (use Kafka/Debezium)

```

Q158. What is pg_cron for scheduled jobs?

Difficulty:
Medium

```

-- pg_cron: schedule jobs within PostgreSQL

CREATE EXTENSION pg_cron;

-- Schedule a job (cron syntax: min hour day month weekday)
SELECT cron.schedule(
    'daily-cleanup',          -- job name
    '0 2 * * *',             -- every day at 2 AM
    'DELETE FROM events WHERE created_at < NOW() - INTERVAL ''90 days'''
);

-- Other examples
SELECT cron.schedule('vacuum-orders', '0 3 * * 0', -- Sundays at 3 AM
    'VACUUM ANALYZE orders');

SELECT cron.schedule('refresh-stats', '* / 5 * * * *', -- every 5 minutes
    'REFRESH MATERIALIZED VIEW CONCURRENTLY mv_order_stats');

-- List scheduled jobs
SELECT * FROM cron.job;

-- View recent job run history
SELECT * FROM cron.job_run_details
ORDER BY start_time DESC LIMIT 20;

-- Unschedule
SELECT cron.unschedule('daily-cleanup');
SELECT cron.unschedule(job_id); -- by ID

-- Configuration (postgresql.conf):
-- cron.database_name = 'mydb' -- which DB runs jobs

```

```
-- shared_preload_libraries = 'pg_cron'
```

Q159. What is PostgreSQL full-text search?

Difficulty: Hard

```
-- Full-text search: search within text fields

-- tsvector: preprocessed document representation
-- tsquery: search query

-- Basic FTS
SELECT * FROM products
WHERE to_tsvector('english', description) @@ to_tsquery('english', 'wireless & headphones');

-- Store tsvector for performance
ALTER TABLE products ADD COLUMN search_vector tsvector;

-- Populate
UPDATE products SET search_vector =
    to_tsvector('english',
        coalesce(name, '') || ' ' ||
        coalesce(description, '') || ' ' ||
        coalesce(brand, ''));

-- GIN index for fast FTS
CREATE INDEX idx_products_search ON products USING GIN(search_vector);

-- Auto-update with trigger
CREATE OR REPLACE FUNCTION update_product_search() RETURNS trigger AS $$
BEGIN
    NEW.search_vector := to_tsvector('english',
        coalesce(NEW.name, '') || ' ' || coalesce(NEW.description, ''));
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER product_search_update
    BEFORE INSERT OR UPDATE ON products
    FOR EACH ROW EXECUTE FUNCTION update_product_search();

-- Search with ranking
SELECT id, name,
    ts_rank(search_vector, query) AS rank,
    ts_headline('english', description, query) AS snippet
FROM products, to_tsquery('english', 'wireless | bluetooth') query
WHERE search_vector @@ query
ORDER BY rank DESC
LIMIT 20;

-- Phrase search
to_tsquery('english', '''wireless headphones''') -- exact phrase

-- OR, AND, NOT
to_tsquery('english', 'wireless & !gaming')      -- wireless AND NOT gaming
to_tsquery('english', 'wireless | bluetooth')     -- OR
```

Q160. What is PostgreSQL Foreign Data Wrappers (FDW)?

Difficulty: Hard

```
-- FDW: query external data sources as if they were local tables

-- postgres_fdw: query another PostgreSQL database
CREATE EXTENSION postgres_fdw;

CREATE SERVER remote_db
    FOREIGN DATA WRAPPER postgres_fdw
    OPTIONS (host 'db2.example.com', port '5432', dbname 'analytics');

CREATE USER MAPPING FOR myuser
    SERVER remote_db
    OPTIONS (user 'analytics_reader', password 'secret');

-- Import all tables from remote schema
IMPORT FOREIGN SCHEMA public
    FROM SERVER remote_db INTO foreign_schema;

-- Or create specific foreign table
CREATE FOREIGN TABLE remote_orders (
    id BIGINT,
    customer_id BIGINT,
    amount DECIMAL(12,2),
    created_at TIMESTAMPTZ
) SERVER remote_db
OPTIONS (schema_name 'public', table_name 'orders');

-- Query as if local (pushes WHERE/JOIN to remote)
SELECT * FROM remote_orders WHERE created_at > NOW() - INTERVAL '7 days';

-- file_fdw: query CSV/text files
CREATE EXTENSION file_fdw;
CREATE SERVER file_server FOREIGN DATA WRAPPER file_fdw;
CREATE FOREIGN TABLE data_import (col1 TEXT, col2 INT, col3 DATE)
    SERVER file_server
    OPTIONS (filename '/data/import.csv', format 'csv', header 'true');

-- Other FDWs: mysql_fdw, mongo_fdw, redis_fdw, S3 (parquet_s3_fdw)
```

Q161. What is PostgreSQL connection pooling with PgBouncer?

Difficulty: Hard

```
; PgBouncer: connection pooler for PostgreSQL
```

```
[databases]
```

```
myapp = host=postgres port=5432 dbname=myapp
```

```
[pgbouncer]
```

```
listen_port = 5432
```

```
listen_addr = 0.0.0.0
```

```
auth_type = scram-sha-256
```

```
auth_file = /etc/pgbouncer/userlist.txt
```

```
; Pool modes:
```

```
; session: connection held for entire client session (compatible, wasteful)
```

```
; transaction: connection returned to pool after each transaction (recommended)
```

```
; statement: connection returned after each statement (breaks multi-statement txns)
```

```
pool_mode = transaction
```

```

default_pool_size = 20 ; connections per database+user combo
max_client_conn = 1000 ; total client connections
min_pool_size = 5 ; keep at least 5 connections

reserve_pool_size = 5 ; extra connections under load
reserve_pool_timeout = 5 ; seconds before using reserve

server_idle_timeout = 600 ; close idle server connections after 10min
client_idle_timeout = 0 ; don't close idle clients

; Monitoring
stats_period = 60
listen_port = 6432 ; admin interface

PgBouncer workflow:
  1000 app connections → PgBouncer → 20 DB connections
  In transaction mode: connection borrowed for duration of transaction

pgbouncer-rr: statement-level load balancing across replicas
Odyssey: alternative (better for high connection counts)

```

Q162. What is PostgreSQL VACUUM internals?

Difficulty: Hard

```

-- VACUUM: reclaims space from dead tuples (MVCC debris)
-- Dead tuples: old versions after UPDATE/DELETE

-- Basic VACUUM (non-blocking)
VACUUM orders;

-- VACUUM ANALYZE (update statistics too)
VACUUM ANALYZE orders;

-- VACUUM FULL: reclaims space, rewrites table (LOCKS table, slow!)
-- Use only when table bloat is severe and downtime acceptable
VACUUM FULL orders;

-- View bloat
SELECT schemaname, tablename,
       n_live_tup, n_dead_tup,
       round(n_dead_tup::numeric / nullif(n_live_tup, 0) * 100, 2) AS dead_pct,
       last_vacuum, last_autovacuum,
       last_analyze, last_autoanalyze
FROM pg_stat_user_tables
ORDER BY n_dead_tup DESC;

-- Autovacuum settings (per table override)
ALTER TABLE hot_table SET (
    autovacuum_vacuum_threshold = 100,      -- vacuum if >100 dead tuples
    autovacuum_vacuum_scale_factor = 0.01,  -- or >1% of table
    autovacuum_vacuum_cost_delay = 2        -- less aggressive (ms)
);

-- Transaction ID wraparound prevention (critical!)
-- PostgreSQL has 4-billion transaction ID limit
-- autovacuum freezes old rows (prevents wraparound)
-- Monitor:
SELECT datname, age(datfrozenxid) as xid_age,

```

```

2^31 - age(datfrozenxid) as remaining
FROM pg_database ORDER BY age(datfrozenxid) DESC;
-- Alert if xid_age > 1.5 billion!

```

Q163. What is PostgreSQL advisory locks?

Difficulty: Hard

```

-- Advisory locks: application-level cooperative locks
-- Not tied to rows/tables (you decide what they protect)
-- Two types: session (held until released/disconnect) or transaction (auto-released)

-- Session-level
SELECT pg_try_advisory_lock(42); -- try (non-blocking): true/false
SELECT pg_advisory_lock(42);      -- blocking: waits until available
SELECT pg_advisory_unlock(42);
SELECT pg_advisory_unlock_all();  -- release all session locks

-- Transaction-level (auto-released at COMMIT/ROLLBACK)
SELECT pg_try_advisory_xact_lock(42);

-- Distributed leader election:
-- Only one pod can acquire lock → it's the leader
func acquireLeadership(ctx context.Context, db *sql.DB, lockID int64) (bool, error) {
    var acquired bool
    err := db.QueryRowContext(ctx,
        "SELECT pg_try_advisory_lock($1)", lockID).Scan(&acquired)
    return acquired, err
}

-- Prevent concurrent job execution:
func runJobSafely(ctx context.Context, db *sql.DB, jobID int64) error {
    var acquired bool
    db.QueryRowContext(ctx, "SELECT pg_try_advisory_lock($1)", jobID).Scan(&acquired)
    if !acquired { return ErrJobAlreadyRunning }
    defer db.ExecContext(ctx, "SELECT pg_advisory_unlock($1)", jobID)

    return runJob(ctx)
}

```

Q164. What is PostgreSQL connection troubleshooting?

Difficulty:
Medium

```

-- View all connections
SELECT pid, username, application_name, client_addr,
       state, wait_event_type, wait_event,
       now() - query_start AS query_age,
       query
FROM pg_stat_activity
WHERE datname = 'mydb'
ORDER BY query_start;

-- Find long-running queries (> 30 seconds)
SELECT pid, now() - query_start AS duration, query, state
FROM pg_stat_activity
WHERE state != 'idle'
AND now() - query_start > INTERVAL '30 seconds'
ORDER BY duration DESC;

```



```
-- Kill a query
SELECT pg_cancel_backend(pid); -- send SIGINT (gentle)
SELECT pg_terminate_backend(pid); -- send SIGTERM (forceful)

-- Kill idle connections older than 10 minutes
SELECT pg_terminate_backend(pid)
FROM pg_stat_activity
WHERE state = 'idle'
AND state_change < NOW() - INTERVAL '10 minutes'
AND pid != pg_backend_pid();

-- Connection limits
SHOW max_connections; -- default 100
SELECT count(*) FROM pg_stat_activity WHERE datname = 'mydb';

-- Set per-database connection limit
ALTER DATABASE mydb CONNECTION LIMIT 50;

-- Set per-user connection limit
ALTER USER myapp CONNECTION LIMIT 20;
```

Q165. What is PostgreSQL streaming replication setup?

Difficulty: Hard

```
# Primary server (postgresql.conf)
wal_level = replica          # enable WAL streaming
max_wal_senders = 5          # connections for replication
max_replication_slots = 5    # replication slots
wal_keep_size = 1GB          # keep WAL for replicas

# pg_hba.conf on primary
host replication replicator replica-ip/32 scram-sha-256

# Create replication user
CREATE USER replicator WITH REPLICATION PASSWORD 'secret';

# Standby: take base backup
pg_basebackup -h primary-host -U replicator -D /var/lib/postgresql/data \
  -P -Xs -R
# -Xs: streaming WAL, -R: create recovery.conf/standby.signal

# Standby (postgresql.conf)
primary_conninfo = 'host=primary-host user=replicator password=secret'
hot_standby = on # allow queries on standby

# Check replication status (on primary)
SELECT * FROM pg_stat_replication;
-- sent_lsn, write_lsn, flush_lsn, replay_lsn, sync_state

# Check lag (on standby)
SELECT now() - pg_last_xact_replay_timestamp() AS replication_delay;

# Replication slots (don't lose WAL even if replica disconnects)
SELECT pg_create_physical_replication_slot('standby1');
-- WARNING: unbounded WAL growth if replica stays down long!
-- Monitor: SELECT * FROM pg_replication_slots;
```

Q166. What is PostgreSQL bulk operations?**Difficulty: Hard**

```
// Bulk insert with COPY (fastest method)
conn, _ := pgx.Connect(ctx, dsn)

rows := [][]interface{}{
    {1, "Alice", "alice@example.com"},
    {2, "Bob", "bob@example.com"},
    // ... thousands more
}

columns := []string{"id", "name", "email"}

n, err := conn.CopyFrom(
    ctx,
    pgx.Identifier{"users"},
    columns,
    pgx.CopyFromRows(rows),
)
// COPY is 10-100x faster than individual INSERTs
// No network round-trips between rows

// Bulk update with unnest
ids := []int64{1, 2, 3}
names := []string{"Alice2", "Bob2", "Charlie2"}

_, err = pool.Exec(ctx, `
    UPDATE users SET name = bulk.name
    FROM unnest($1::bigint[], $2::text[]) AS bulk(id, name)
    WHERE users.id = bulk.id
`, ids, names)

// Bulk upsert with unnest
_, err = pool.Exec(ctx, `
    INSERT INTO products (id, name, price)
    SELECT * FROM unnest($1::bigint[], $2::text[], $3::decimal[])
    ON CONFLICT (id) DO UPDATE
    SET name = EXCLUDED.name, price = EXCLUDED.price
`, ids, names, prices)
```

Q167. What is PostgreSQL query optimization workflow?**Difficulty: Hard**

```
-- Step 1: Find slow queries
SELECT query, calls, total_exec_time/calls AS avg_ms,
       rows/calls AS avg_rows,
       shared_blks_hit / (shared_blks_hit + shared_blks_read + 0.001) AS cache_hit
FROM pg_stat_statements
WHERE calls > 100
ORDER BY total_exec_time DESC
LIMIT 20;

-- Step 2: EXPLAIN the slow query
EXPLAIN (ANALYZE, BUFFERS, FORMAT TEXT)
SELECT o.*, c.name
FROM orders o
JOIN customers c ON c.id = o.customer_id
```

```

WHERE o.status = 'pending'
AND o.created_at > NOW() - INTERVAL '7 days';

-- Step 3: Identify issues
-- Seq Scan on large table → missing index
-- Nested Loop + many rows → consider Hash Join
-- High Buffers hit → memory pressure
-- Large Sort → work_mem increase or index for ORDER BY

-- Step 4: Add index
CREATE INDEX CONCURRENTLY idx_orders_status_created
ON orders(status, created_at DESC)
WHERE status = 'pending'; -- partial index for hot path

-- Step 5: Verify improvement
EXPLAIN (ANALYZE, BUFFERS)
-- run query again → should show Index Scan

-- Common query patterns and fixes:
-- N+1: use JOIN or batch with IN (...)
-- LIKE '%suffix%': use pg_trgm + GIN index
-- JSON queries: use GIN index on jsonb column
-- Range queries: BRIN index for sequential data
-- Full table scan: check pg_stats, increase statistics target

```

Q168. What is PostgreSQL monitoring queries?

Difficulty:
Medium

```

-- Database size
SELECT pg_database_size('mydb') / 1024 / 1024 AS size_mb;

-- Table sizes (with bloat estimate)
SELECT schemaname, tablename,
       pg_size_pretty(pg_total_relation_size(schemaname||'.'||tablename)) AS total_size,
       pg_size_pretty(pg_relation_size(schemaname||'.'||tablename)) AS table_size,
       pg_size_pretty(pg_total_relation_size(schemaname||'.'||tablename) -
                    pg_relation_size(schemaname||'.'||tablename)) AS index_size
FROM pg_tables
WHERE schemaname = 'public'
ORDER BY pg_total_relation_size(schemaname||'.'||tablename) DESC
LIMIT 20;

-- Index usage (find unused indexes)
SELECT schemaname, tablename, indexname,
       idx_scan, idx_tup_read, idx_tup_fetch,
       pg_size_pretty(pg_relation_size(indexrelid)) AS index_size
FROM pg_stat_user_indexes
ORDER BY idx_scan ASC;
-- idx_scan = 0 → index never used → candidate for removal

-- Cache hit ratio (should be >99%)
SELECT
       sum(blks_hit) / (sum(blks_hit) + sum(blks_read) + 0.001) AS hit_ratio
FROM pg_stat_database
WHERE datname = 'mydb';

-- Locks waiting

```

```
SELECT * FROM pg_locks WHERE granted = false;

-- Replication lag
SELECT client_addr, state,
       pg_wal_lsn_diff(pg_current_wal_lsn(), sent_lsn) AS send_lag_bytes,
       pg_wal_lsn_diff(sent_lsn, replay_lsn) AS replay_lag_bytes
FROM pg_stat_replication;
```

Q169. What is PostgreSQL error handling in Go?

Difficulty:
Medium

```
import (
    "github.com/jackc/pgx/v5/pgconn"
    "errors"
)

// Handle specific PostgreSQL errors
func createUser(ctx context.Context, pool *pgxpool.Pool, user User) error {
    _, err := pool.Exec(ctx,
        "INSERT INTO users(email, name) VALUES($1, $2)",
        user.Email, user.Name)

    if err == nil { return nil }

    // Check PostgreSQL error code
    var pgErr *pgconn.PgError
    if errors.As(err, &pgErr) {
        switch pgErr.Code {
            case "23505": // unique_violation
                return fmt.Errorf("user with email %s already exists", user.Email)
            case "23503": // foreign_key_violation
                return fmt.Errorf("referenced entity does not exist")
            case "23502": // not_null_violation
                return fmt.Errorf("required field %s is missing", pgErr.ColumnName)
            case "42P01": // undefined_table
                return fmt.Errorf("table %s does not exist", pgErr.TableName)
            case "40001": // serialization_failure (retry!)
                return ErrSerializationFailure
            case "40P01": // deadlock_detected (retry!)
                return ErrDeadlock
        }
        return fmt.Errorf("db error %s: %s", pgErr.Code, pgErr.Message)
    }
    return err
}

// PostgreSQL error codes (SQLSTATE)
// 23xxx: integrity constraint violations
// 40xxx: transaction rollback (retry-able)
// 42xxx: syntax error or access rule violation
// 08xxx: connection exception
// Full list: https://www.postgresql.org/docs/current/errcodes-appendix.html
```

Q170. What is PostgreSQL JSON querying?

Difficulty:
Medium

```
-- JSONB: binary JSON (indexed, faster queries)
```

```

-- JSON: text JSON (preserves whitespace/order)

CREATE TABLE events (
    id BIGSERIAL PRIMARY KEY,
    data JSONB NOT NULL
);

-- Insert
INSERT INTO events(data) VALUES
    ('{"type":"click","user":{"id":42,"name":"Alice"},"page":"/products","ts":1234}');

-- Query operators
data->>'type'          -- extract text value
data->'user'            -- extract JSON value
data->'user'->>'name'    -- nested access
data#>>'{user,name}'    -- path access

-- WHERE conditions
WHERE data->>'type' = 'click'
WHERE data->'user'->>'id' = '42'
WHERE data ? 'page'          -- key exists
WHERE data ?| ARRAY['click', 'purchase'] -- any key exists
WHERE data ?& ARRAY['type', 'user']    -- all keys exist
WHERE data @> '{"type":"click"}'::jsonb -- contains

-- GIN indexes for JSONB
CREATE INDEX idx_events_data ON events USING GIN(data); -- all keys
CREATE INDEX idx_events_type ON events USING GIN((data->'type')); -- specific

-- Aggregation
SELECT data->>'type' AS event_type, count(*)
FROM events
GROUP BY data->>'type';

-- Update JSONB
UPDATE events SET data = data || '{"processed":true}'
WHERE id = 1;

UPDATE events SET data = jsonb_set(data, '{user,verified}', 'true')
WHERE id = 1;

```

Q171. ### Q171-Q200: Final PostgreSQL Questions

Difficulty:
Medium

Q	Topic
Q171	pgx v5 batch mode queries
Q172	PostgreSQL connection string configuration
Q173	PostgreSQL role-based access control
Q174	Row-level security policies
Q175	PostgreSQL table inheritance
Q176	PostgreSQL domain types and constraints
Q177	Materialized views refresh strategies
Q178	PostgreSQL EXPLAIN ANALYZE output reading

Q179	pg_stat_statements configuration
Q180	PostgreSQL hot standby queries
Q181	Synchronous vs asynchronous commit
Q182	PostgreSQL checkpoint configuration
Q183	WAL archiving and PITR
Q184	Barman for PostgreSQL backup
Q185	pgBackRest for backup and restore
Q186	PostgreSQL high availability with Patroni
Q187	Connection pooling: PgBouncer vs Odyssey
Q188	PostgreSQL table statistics and analyze
Q189	PostgreSQL bloat measurement and management
Q190	Extensions: pg_stat_statements, pg_buffercache
Q191	PostgreSQL unlogged tables
Q192	PostgreSQL sequences and serial types
Q193	Time-series with TimescaleDB extension
Q194	Citus for horizontal sharding
Q195	PostgreSQL logical replication for CDC
Q196	sqlc code generation from SQL
Q197	PostgreSQL interview: design order management schema
Q198	PostgreSQL interview: optimize slow aggregation query
Q199	PostgreSQL interview: handle high-concurrency upsert
Q200	PostgreSQL SDE2 production checklist

Extended Questions (Q172–Q200)

Q172. What is PostgreSQL logical replication?

Difficulty: Hard

```
-- Logical replication: replicate specific tables (not entire WAL stream)
-- Use: zero-downtime migrations, selective sync, multi-master

-- Publisher (source)
CREATE PUBLICATION my_pub FOR TABLE users, orders, products;
-- Or all tables:
CREATE PUBLICATION my_pub FOR ALL TABLES;

-- Subscriber (target)
CREATE SUBSCRIPTION my_sub
  CONNECTION 'host=source-db port=5432 dbname=mydb user=replicator password=secret'
  PUBLICATION my_pub;
```

```
-- Check status
SELECT * FROM pg_stat_replication;
SELECT * FROM pg_replication_slots;
SELECT * FROM pg_stat_subscription;

-- Requirements:
-- wal_level = logical (source)
-- Tables must have primary keys (or REPLICATION IDENTITY FULL)
ALTER TABLE users REPLICATION IDENTITY FULL; -- if no PK

-- Use case: migrate to new schema
-- 1. Set up logical replication to new DB
-- 2. Let it catch up (near-zero lag)
-- 3. Switch application to new DB
-- 4. Zero downtime!
```

Q173. What is PostgreSQL connection pooling with PgBouncer?

Difficulty: Hard

```
# PgBouncer: connection pooler between app and PostgreSQL
# PostgreSQL max_connections (default 100) is expensive (~10MB/connection)
# PgBouncer: many app connections → few DB connections
```

```
# pgbouncer.ini
[databases]
mydb = host=postgres port=5432 dbname=mydb

[pgbouncer]
listen_addr = 0.0.0.0
listen_port = 5432
auth_type = md5
auth_file = /etc/pgbouncer/userlist.txt
pool_mode = transaction      # transaction-level pooling (recommended)
max_client_conn = 1000      # app connections
default_pool_size = 20      # DB connections per database/user pair
min_pool_size = 5
reserve_pool_size = 5
server_lifetime = 3600      # close idle server connection after 1h
server_idle_timeout = 600   # 10 min idle timeout
```

```
Pool modes:
  session:    connection held for entire session (like no pooler)
  transaction: connection returned after each transaction (recommended!)
               ■■ No: LISTEN/NOTIFY, prepared statements, SET vars
  statement:  return after each statement (very aggressive, rarely used)
```

```
With transaction mode:
  1000 app connections → 20 DB connections
  90%+ reduction in PostgreSQL memory usage!
```

```
pgBouncer vs pgpool-II:
  PgBouncer: simple pooling, high performance, lower features
  pgpool-II: pooling + load balancing + failover, more complex
```

Q174. What is PostgreSQL TOAST (The Oversized-Attribute Storage Technique)?

Difficulty: Hard

```
-- TOAST: how PostgreSQL stores large values (> ~2KB)
-- Transparent: you don't see it, but it affects performance
```

```
-- TOAST strategies per column:
-- PLAIN:      no compression, no out-of-line storage (int, bool, etc.)
-- MAIN:       try to compress first, then out-of-line if needed
-- EXTENDED:   try both (default for text, bytea, json)
-- EXTERNAL:   out-of-line, no compression (fast read of substrings)

-- Check TOAST strategy
SELECT attname, attstorage FROM pg_attribute
WHERE attrelid = 'my_table'::regclass AND attnum > 0;

-- Change strategy
ALTER TABLE docs ALTER COLUMN content SET STORAGE EXTERNAL;
-- Use EXTERNAL for large text where you frequently use LIKE/POSITION

-- TOAST tables (internal)
SELECT relname FROM pg_class WHERE reltoastrelid = 'my_table'::regclass::oid;

-- Performance: large JSONB columns
-- Problem: fetching one small field fetches entire TOAST value
-- Solution: separate frequently-accessed small fields
ALTER TABLE events ADD COLUMN event_type text; -- separate column
-- Don't embed in JSONB if queried frequently as filter
```

Q175. What is PostgreSQL table partitioning strategies?

Difficulty: Hard

```
-- Range partitioning (most common for time-series)
CREATE TABLE events (
    id          bigserial,
    occurred_at timestamptz NOT NULL,
    type        text,
    payload     jsonb
) PARTITION BY RANGE (occurred_at);

-- Monthly partitions
CREATE TABLE events_2024_01 PARTITION OF events
    FOR VALUES FROM ('2024-01-01') TO ('2024-02-01');

CREATE TABLE events_2024_02 PARTITION OF events
    FOR VALUES FROM ('2024-02-01') TO ('2024-03-01');

-- Default partition (catch-all)
CREATE TABLE events_default PARTITION OF events DEFAULT;

-- Auto partition creation with pg_partman
SELECT partman.create_parent('public.events', 'occurred_at', 'native', 'monthly');

-- List partitioning (for enum-like values)
CREATE TABLE orders (
    id          bigserial,
    status text,
    amount numeric
) PARTITION BY LIST (status);

CREATE TABLE orders_pending PARTITION OF orders FOR VALUES IN ('pending');
CREATE TABLE orders_complete PARTITION OF orders FOR VALUES IN ('completed', 'shipped');
CREATE TABLE orders_cancelled PARTITION OF orders FOR VALUES IN ('cancelled');
```



```
-- Hash partitioning (for even distribution without time-based access)
CREATE TABLE users (id bigserial, email text) PARTITION BY HASH (id);
CREATE TABLE users_0 PARTITION OF users FOR VALUES WITH (modulus 4, remainder 0);
CREATE TABLE users_1 PARTITION OF users FOR VALUES WITH (modulus 4, remainder 1);
CREATE TABLE users_2 PARTITION OF users FOR VALUES WITH (modulus 4, remainder 2);
CREATE TABLE users_3 PARTITION OF users FOR VALUES WITH (modulus 4, remainder 3);
```

Q176. What is PostgreSQL WAL (Write-Ahead Log) internals?

Difficulty: Hard

```
-- WAL: all changes written to log first, then data files
-- Ensures durability (fsync WAL = durable, even if crash)

-- WAL configuration
wal_level = replica          -- minimal | replica | logical
max_wal_size = 1GB          -- trigger checkpoint when WAL exceeds this
min_wal_size = 80MB         -- keep at least this much WAL
checkpoint_completion_target = 0.9 -- spread checkpoint over 90% of interval
wal_compression = on        -- compress WAL records (reduces I/O)
wal_buffers = 16MB          -- WAL buffer in shared memory

-- WAL segments: 16MB files in pg_wal/
-- LSN (Log Sequence Number): position in WAL stream
SELECT pg_current_wal_lsn();
SELECT pg_walfile_name(pg_current_wal_lsn());

-- WAL archiving (for PITR)
archive_mode = on
archive_command = 'cp %p /wal_archive/%f'
-- or: aws s3 cp %p s3://my-bucket/wal/%f

-- Point-in-time recovery
-- Restore base backup → replay WAL up to target time
recovery_target_time = '2024-01-15 10:30:00'
recovery_target_action = 'promote'

-- Monitor WAL lag
SELECT client_addr, state, sent_lsn, write_lsn, flush_lsn, replay_lsn,
       (sent_lsn - replay_lsn) AS replication_lag_bytes
FROM pg_stat_replication;
```

Q177. What is PostgreSQL vacuum and autovacuum tuning?

Difficulty: Hard

```
-- MVCC creates dead tuples → VACUUM removes them
-- AUTOVACUUM: runs automatically, but may need tuning

-- Check bloat
SELECT relname, n_live_tup, n_dead_tup,
       round(n_dead_tup::numeric/nullif(n_live_tup,0)*100, 2) AS dead_pct,
       last_autovacuum, last_autoanalyze
FROM pg_stat_user_tables
ORDER BY n_dead_tup DESC;

-- Check autovacuum settings for table
SELECT reloptions FROM pg_class WHERE relname = 'orders';

-- Tune autovacuum per table (high-write tables need aggressive settings)
```

```

ALTER TABLE orders SET (
    autovacuum_vacuum_scale_factor = 0.01,    -- trigger at 1% dead tuples (vs 20%)
    autovacuum_analyze_scale_factor = 0.005,  -- analyze at 0.5% changed
    autovacuum_vacuum_cost_delay = 2,         -- less delay = faster vacuum
    autovacuum_vacuum_cost_limit = 400        -- higher limit = faster vacuum
);

-- Manual vacuum (when autovacuum can't keep up)
VACUUM (VERBOSE, ANALYZE) orders;
VACUUM FULL orders;  -- rewrites table, reclaims space, LOCKS TABLE!

-- Transaction ID wraparound (critical!)
-- Every table needs VACUUM before XID reaches 2^31
-- Monitor: pg_database.datfrozenxid
SELECT datname, age(datfrozenxid) as xid_age FROM pg_database ORDER BY xid_age DESC;
-- Alert if xid_age > 1.5 billion (approaching wraparound)

```

Q178. What is PostgreSQL full-text search?

Difficulty:
Medium

```

-- tsvector: preprocessed document
-- tsquery: search query

-- Basic full-text search
SELECT * FROM articles
WHERE to_tsvector('english', title || ' ' || body) @@ to_tsquery('postgresql & performance');

-- Better: indexed column
ALTER TABLE articles ADD COLUMN search_vector tsvector;

UPDATE articles SET search_vector =
    to_tsvector('english', coalesce(title,'') || ' ' || coalesce(body,''));

CREATE INDEX articles_search_idx ON articles USING GIN(search_vector);

-- Keep updated with trigger
CREATE FUNCTION update_search_vector() RETURNS trigger AS $$
BEGIN
    NEW.search_vector := to_tsvector('english', coalesce(NEW.title,'') || ' ' || coalesce(NEW.body,''));
    RETURN NEW;
END $$ LANGUAGE plpgsql;

CREATE TRIGGER articles_search_update
    BEFORE INSERT OR UPDATE ON articles
    FOR EACH ROW EXECUTE FUNCTION update_search_vector();

-- Search with ranking
SELECT title, ts_rank(search_vector, query) AS rank
FROM articles, to_tsquery('english', 'postgres & index') query
WHERE search_vector @@ query
ORDER BY rank DESC
LIMIT 10;

-- Highlighting
SELECT ts_headline('english', body, to_tsquery('postgresql'), 'MaxWords=50') FROM articles;

-- For production FTS: consider Elasticsearch/OpenSearch (better ranking, scaling)

```

Q179. What is PostgreSQL LISTEN/NOTIFY?**Difficulty: Medium**

```
// LISTEN/NOTIFY: real-time notifications from PostgreSQL
// Use: cache invalidation, event streaming, live dashboard updates

// SQL side: NOTIFY
// Trigger: notify on row change
CREATE OR REPLACE FUNCTION notify_changes() RETURNS trigger AS $$
BEGIN
    PERFORM pg_notify(
        'table_changes',
        json_build_object(
            'table', TG_TABLE_NAME,
            'action', TG_OP,
            'id', NEW.id
        )::text
    );
    RETURN NEW;
END $$ LANGUAGE plpgsql;

CREATE TRIGGER after_order_change
    AFTER INSERT OR UPDATE ON orders
    FOR EACH ROW EXECUTE FUNCTION notify_changes();

// Go side: listen for notifications (pgx)
conn, _ := pgx.Connect(ctx, dsn)
_, err = conn.Exec(ctx, "LISTEN table_changes")

for {
    notification, err := conn.WaitForNotification(ctx)
    if err != nil { break }

    log.Printf("channel=%s payload=%s", notification.Channel, notification.Payload)
    invalidateCache(notification.Payload)
}

// Limitations:
// Payload max: 8000 bytes
// No persistence (if listener disconnected → events lost)
// Not a message queue (use Kafka/RMQ for that)
// Great for: cache invalidation, real-time UI updates
```

Q180. What is PostgreSQL row-level security (RLS)?**Difficulty: Hard**

```
-- RLS: filter rows based on current user context
-- Use: multi-tenant apps (each tenant sees only their data)

-- Enable RLS
ALTER TABLE orders ENABLE ROW LEVEL SECURITY;

-- Policy: users see only their tenant's data
CREATE POLICY tenant_isolation ON orders
    USING (tenant_id = current_setting('app.tenant_id')::bigint);

-- In application: set tenant context before queries
```

```

SET LOCAL app.tenant_id = '42';
SELECT * FROM orders;  -- only returns tenant 42's orders

-- With pgx (Go)
conn.Exec(ctx, "SET LOCAL app.tenant_id = $1", tenantID)

-- Admin bypass
ALTER TABLE orders FORCE ROW LEVEL SECURITY;
CREATE POLICY admin_all ON orders TO admin_role USING (true);

-- Performance: RLS adds predicate to every query
-- Indexes must cover tenant_id:
CREATE INDEX orders_tenant_id_idx ON orders(tenant_id);
-- Or composite: (tenant_id, created_at) for range queries

-- Verify RLS is working
EXPLAIN SELECT * FROM orders;
-- Should show: Filter: (tenant_id = current_setting('app.tenant_id')...)

```

Q181. What is PostgreSQL advisory locks?

Difficulty:
Medium

```

-- Advisory locks: application-level locking (not table/row locks)
-- Stored in shared memory, not in tables
-- Use: distributed cron jobs, leader election, resource limiting

-- Session-level advisory lock (held until session ends or unlocked)
SELECT pg_try_advisory_lock(12345);  -- try, non-blocking
SELECT pg_advisory_lock(12345);      -- block until acquired
SELECT pg_advisory_unlock(12345);    -- release

-- Transaction-level advisory lock (auto-released at transaction end)
SELECT pg_try_advisory_xact_lock(12345);  -- in transaction

-- Prevent multiple cron job instances (distributed lock)
-- Go example:
func runCronJob(ctx context.Context, db *pgxpool.Pool, jobID int64) error {
    conn, _ := db.Acquire(ctx)
    defer conn.Release()

    var locked bool
    conn.QueryRow(ctx, "SELECT pg_try_advisory_lock($1)", jobID).Scan(&locked)
    if !locked {
        return nil // another instance is running
    }
    defer conn.Exec(ctx, "SELECT pg_advisory_unlock($1)", jobID)

    return doWork(ctx)
}

-- Named locks (use hash of string)
SELECT pg_try_advisory_lock(hashtext('my-unique-job-name'));

-- List held advisory locks
SELECT * FROM pg_locks WHERE locktype = 'advisory';

```

Q182. What is PostgreSQL explain analyze interpretation?**Difficulty: Hard**

```
-- EXPLAIN ANALYZE: actual execution plan + timing
EXPLAIN (ANALYZE, BUFFERS, FORMAT TEXT)
SELECT u.name, count(o.id) as order_count
FROM users u
LEFT JOIN orders o ON o.user_id = u.id
WHERE u.created_at > NOW() - INTERVAL '30 days'
GROUP BY u.id, u.name
ORDER BY order_count DESC
LIMIT 10;
```

-- Reading output:

-- Node types:

- Seq Scan: full table scan (bad for large tables)
- Index Scan: uses index, fetches rows from heap
- Index Only Scan: all data in index (fastest!)
- Bitmap Heap Scan: batches heap access (for moderate selectivity)
- Hash Join: build hash table from smaller table, probe with larger
- Merge Join: both sides sorted, merge (good for large sorted inputs)
- Nested Loop: for each outer row, scan inner (good for small inputs)

-- Key numbers:

- (cost=start..total rows=N width=W)
- Actual time=start..actual (ms per execution)
- Rows: estimated vs actual (large discrepancy = stale statistics)
- Buffers: shared hit=X read=Y (read from disk vs cache)

-- Red flags:

- Rows: 1000 estimated vs 100000 actual → run ANALYZE
- Seq Scan on large table with WHERE clause → add index
- Hash Batches > 1 → increase work_mem
- Nested Loop with many iterations → rewrite or add index

Q183. What is PostgreSQL connection limits and pooling best practices?**Difficulty: Hard**

```
-- max_connections (default: 100)
-- Each connection: ~5-10MB shared memory + backend process
-- With 100 connections: 500MB-1GB RAM just for connections!
```

-- Check current connections

```
SELECT count(*), state, wait_event_type, wait_event
FROM pg_stat_activity
GROUP BY state, wait_event_type, wait_event;
```

-- Terminate idle connections (emergency)

```
SELECT pg_terminate_backend(pid)
FROM pg_stat_activity
WHERE state = 'idle'
AND state_change < NOW() - INTERVAL '10 minutes';
```

-- Connection limits per role

```
CREATE ROLE api_user CONNECTION LIMIT 50;
ALTER ROLE api_user CONNECTION LIMIT 100;
```

-- Architecture recommendation:

```
-- Small DB (< 10 app servers): max_connections = 100-200
-- Medium DB: max_connections = 200-500 + PgBouncer
-- Large DB: PgBouncer with transaction pooling

-- PgBouncer sizing:
-- app_threads × app_servers / avg_query_time_ms = required_db_connections
-- 10 threads × 20 servers / 10ms = 20 connections needed!
-- (much less than 200 app "connections")

-- max_connections in postgresql.conf
ALTER SYSTEM SET max_connections = 200;
SELECT pg_reload_conf(); -- (or restart for max_connections)
```

Q184. What is PostgreSQL foreign data wrappers (FDW)?

Difficulty:
Medium

```
-- FDW: query external data sources as if they were tables
-- Use: data federation, legacy system integration

-- postgres_fdw: query another PostgreSQL database
CREATE EXTENSION postgres_fdw;

CREATE SERVER remote_db FOREIGN DATA WRAPPER postgres_fdw
    OPTIONS (host 'other-db.example.com', port '5432', dbname 'analytics');

CREATE USER MAPPING FOR current_user
    SERVER remote_db OPTIONS (user 'analytics_user', password 'secret');

CREATE FOREIGN TABLE remote_events (
    id          bigint,
    event_type  text,
    occurred_at timestampz
) SERVER remote_db OPTIONS (schema_name 'public', table_name 'events');

-- Query as if local
SELECT * FROM remote_events WHERE occurred_at > NOW() - INTERVAL '1 day';
-- Predicate pushdown: WHERE clause sent to remote DB!

-- Other FDWs:
-- file_fdw: CSV/text files as tables
-- mysql_fdw: MySQL
-- redis_fdw: Redis
-- s3_fdw: AWS S3 files (Parquet, CSV)
-- tds_fdw: SQL Server / Sybase
-- mongo_fdw: MongoDB

-- Performance: FDW adds network round-trip
-- Good for: occasional queries, not hot paths
```

Q185. What is PostgreSQL statistics and query planner?

Difficulty: **Hard**

```
-- Statistics: pg_statistic (per column) → query planner uses these

-- Update statistics
ANALYZE my_table;
ANALYZE VERBOSE my_table; -- verbose output
```

```
-- Check statistics quality
SELECT attname, n_distinct, correlation,
       array_length(most_common_vals, 1) as mcv_count
FROM pg_stats
WHERE tablename = 'orders';

-- default_statistics_target = 100 (number of MCVs per column)
-- Increase for high-cardinality columns with skewed distribution
ALTER TABLE orders ALTER COLUMN status SET STATISTICS 500;

-- Extended statistics (correlations between columns)
CREATE STATISTICS orders_stats(dependencies) ON status, region FROM orders;
ANALYZE orders;
-- Now planner knows: status='shipped' AND region='us-east' is correlated

-- Force specific plan (for debugging, not production)
SET enable_seqscan = off;      -- force index usage
SET enable_hashjoin = off;    -- force merge or nested loop
SET enable_nestloop = off;    -- force hash or merge join

-- pg_hint_plan (extension): production query hints
/*+ IndexScan(orders orders_user_id_idx) */
SELECT * FROM orders WHERE user_id = 42;
```

Q186. What is PostgreSQL citus for horizontal scaling?

Difficulty: Hard

```
-- Citus: distribute PostgreSQL horizontally across multiple nodes
-- Converts single PostgreSQL node into distributed cluster

-- Install (as PostgreSQL extension)
CREATE EXTENSION citus;

-- Distribute table by shard key (hash-based sharding)
SELECT create_distributed_table('orders', 'tenant_id');
-- Orders now sharded across worker nodes by tenant_id

-- Co-locate related tables (same shard key → same node)
SELECT create_distributed_table('order_items', 'tenant_id',
                                colocate_with => 'orders');
-- Joins between orders and order_items are LOCAL (no network hop)!

-- Reference tables (small tables replicated to all workers)
SELECT create_reference_table('product_catalog');
-- All workers have full copy → local joins against distributed tables

-- Add worker nodes
SELECT citus_add_node('worker1.example.com', 5432);
SELECT citus_add_node('worker2.example.com', 5432);

-- Check distribution
SELECT * FROM citus_shards;
SELECT * FROM citus_stat_statements;

-- Query: coordinator routes to correct shard(s)
SELECT * FROM orders WHERE tenant_id = 42; -- goes to ONE shard
SELECT count(*) FROM orders; -- aggregated across ALL shards (parallel!)
```

Q187. What is PostgreSQL index types comparison?**Difficulty: Hard**

```
-- B-Tree (default): ordered tree, supports <, <=, =, >=, >, BETWEEN, LIKE prefix
CREATE INDEX idx ON users(email); -- default B-Tree
-- Good for: equality, range queries, ORDER BY, most use cases

-- Hash: equality only (=)
CREATE INDEX idx ON users USING HASH (email);
-- Faster for equality than B-Tree, but no range queries

-- GIN (Generalized Inverted Index): for multi-valued types
CREATE INDEX idx ON articles USING GIN (search_vector); -- FTS
CREATE INDEX idx ON products USING GIN (tags);          -- array contains
CREATE INDEX idx ON events USING GIN (payload);          -- JSONB operators
-- @>, ?, ?|, ?&, @@

-- GiST (Generalized Search Tree): geometric, range types
CREATE INDEX idx ON locations USING GiST (coordinates); -- PostGIS
CREATE INDEX idx ON bookings USING GiST (daterange);    -- overlap queries
-- &&, @>, <@, >>

-- BRIN (Block Range Index): for naturally ordered data (timestamps)
CREATE INDEX idx ON events USING BRIN (occurred_at);    -- very small index
-- Only works when data is ordered by insertion (time-series)
-- 1000x smaller than B-Tree, but less precise

-- SP-GiST: space-partitioned GiST (quad-trees, k-d trees)
CREATE INDEX idx ON points USING SPGiST (location);    -- nearest neighbor

-- Bloom filter index: multiple column equality
CREATE INDEX idx ON users USING BLOOM (first_name, last_name, age)
    WITH (length=80, col1=2, col2=2, col3=1);
```

Q188. What is PostgreSQL window functions?**Difficulty: Medium**

```
-- Window functions: compute across related rows without grouping

-- Ranking
SELECT
    name, department, salary,
    RANK() OVER (PARTITION BY department ORDER BY salary DESC) as dept_rank,
    DENSE_RANK() OVER (PARTITION BY department ORDER BY salary DESC) as dense_rank,
    ROW_NUMBER() OVER (PARTITION BY department ORDER BY salary DESC) as row_num,
    NTILE(4) OVER (ORDER BY salary) as quartile
FROM employees;

-- Running totals
SELECT
    date, amount,
    SUM(amount) OVER (ORDER BY date) as running_total,
    AVG(amount) OVER (ORDER BY date ROWS BETWEEN 6 PRECEDING AND CURRENT ROW) as 7day_avg
FROM sales;

-- Lead/Lag (access adjacent rows)
SELECT
    date, amount,
```



```

LAG(amount, 1) OVER (ORDER BY date) as prev_day,
LEAD(amount, 1) OVER (ORDER BY date) as next_day,
amount - LAG(amount, 1) OVER (ORDER BY date) as day_change
FROM sales;

-- First/Last value in window
SELECT
    customer_id, order_date, amount,
    FIRST_VALUE(order_date) OVER (PARTITION BY customer_id ORDER BY order_date) as first_order,
    LAST_VALUE(amount) OVER (PARTITION BY customer_id ORDER BY order_date
        ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING) as last_amount
FROM orders;

```

Q189. What is PostgreSQL pgvector extension?

Difficulty: Hard

```

-- pgvector: store and query vector embeddings (ML/AI use case)
-- Use: semantic search, recommendation, image similarity

CREATE EXTENSION vector;

-- Create table with vector column
CREATE TABLE documents (
    id          bigserial PRIMARY KEY,
    content text,
    embedding vector(1536) -- OpenAI ada-002 dimension
);

-- Insert with embedding
INSERT INTO documents (content, embedding)
VALUES ('PostgreSQL is a relational database', '[0.1, 0.2, ..., 0.9]');

-- Create index for approximate nearest neighbor (ANN) search
-- IVFFlat: partition vectors into lists
CREATE INDEX ON documents USING ivfflat (embedding vector_l2_ops) WITH (lists = 100);

-- HNSW (faster queries, slower build, more memory)
CREATE INDEX ON documents USING hnsw (embedding vector_l2_ops) WITH (m = 16, ef_construction = 64);

-- Search: find 5 most similar documents
SELECT content, embedding <-> '[0.1, 0.2, ...]' AS distance
FROM documents
ORDER BY embedding <-> '[0.1, 0.2, ...]' -- L2 distance
LIMIT 5;

-- Operators:
-- <->: L2 (Euclidean) distance
-- <#>: inner product (for normalized vectors)
-- <=>: cosine distance

-- Hybrid search: combine vector + keyword
SELECT content, embedding <=> $1 as score
FROM documents
WHERE search_vector @@ to_tsquery('postgresql')
ORDER BY embedding <=> $1
LIMIT 10;

```

Q190. What is PostgreSQL performance tuning checklist?**Difficulty: Hard**

```
-- Memory settings
shared_buffers = 25% of RAM          -- main DB cache
effective_cache_size = 75% of RAM    -- hint to planner
work_mem = RAM / max_connections / 4 -- per-sort/hash operation
maintenance_work_mem = 1GB          -- for VACUUM, CREATE INDEX
wal_buffers = 64MB                  -- WAL buffer

-- I/O settings
random_page_cost = 1.1              -- SSD (vs 4.0 default for HDD)
effective_io_concurrency = 200      -- SSD parallel I/O
max_parallel_workers_per_gather = 4

-- Checkpoint settings
checkpoint_completion_target = 0.9
max_wal_size = 4GB

-- Connection settings
max_connections = 200               -- use PgBouncer for more

-- Indexes to check
-- Missing indexes: pg_stat_user_tables.seq_scan high
-- Unused indexes: pg_stat_user_indexes.idx_scan = 0
SELECT indexrelname, idx_scan FROM pg_stat_user_indexes
WHERE idx_scan = 0 AND schemaname = 'public';

-- Slow queries (pg_stat_statements)
SELECT query, calls, mean_exec_time, total_exec_time
FROM pg_stat_statements
ORDER BY total_exec_time DESC LIMIT 20;

-- Table bloat
SELECT tablename, pg_size_pretty(pg_total_relation_size(tablename::regclass))
FROM pg_tables WHERE schemaname = 'public'
ORDER BY pg_total_relation_size(tablename::regclass) DESC;
```

Q191. ### Q191–Q200: Final PostgreSQL Questions**Difficulty: Medium****Q192. What is PostgreSQL pg_stat_statements?****Difficulty: Medium**

```
CREATE EXTENSION pg_stat_statements;
-- Track execution stats for all queries

SELECT query, calls,
       round(mean_exec_time::numeric, 2) as avg_ms,
       round(total_exec_time::numeric, 2) as total_ms,
       round(stddev_exec_time::numeric, 2) as stddev_ms,
       rows / nullif(calls, 0) as rows_per_call
FROM pg_stat_statements
WHERE calls > 100
ORDER BY total_exec_time DESC
LIMIT 20;

-- Reset stats
```

```
SELECT pg_stat_statements_reset();
```

Q193. What is PostgreSQL streaming replication setup?

Difficulty:
Medium

```
# Primary postgresql.conf:
wal_level = replica
max_wal_senders = 5
wal_keep_size = 1GB # keep WAL for lagging replicas

# Primary pg_hba.conf:
host replication replicator 10.0.0.0/8 md5

# Replica setup:
pg_basebackup -h primary -U replicator -D /data/postgres -P -Xs -R
# -R: create standby.signal + postgresql.auto.conf

# Replica postgresql.conf (auto-configured):
primary_conninfo = 'host=primary user=replicator ...'

# Monitor lag:
SELECT pg_wal_lsn_diff(pg_current_wal_lsn(), replay_lsn) AS lag_bytes
FROM pg_stat_replication;
```

Q194. What is PostgreSQL deadlock detection?

Difficulty:
Medium

```
-- PostgreSQL auto-detects deadlocks (deadlock_timeout = 1s)
-- ERROR: deadlock detected
-- DETAIL: Process X waits for ShareLock on transaction Y
-- HINT: See server log for query details.

-- Prevent deadlocks:
-- 1. Always acquire locks in same order
-- 2. Use SELECT ... FOR UPDATE with ORDER BY
-- 3. Minimize transaction duration

-- Detect deadlock patterns in logs:
SELECT query, wait_event, wait_event_type, state
FROM pg_stat_activity
WHERE wait_event_type = 'Lock';

-- View current lock waits
SELECT blocked.pid, blocked.query,
       blocking.pid AS blocking_pid, blocking.query AS blocking_query
FROM pg_stat_activity blocked
JOIN pg_stat_activity blocking ON blocking.pid = ANY(pg_blocking_pids(blocked.pid))
WHERE cardinality(pg_blocking_pids(blocked.pid)) > 0;
```

Q195. What is PostgreSQL jsonb operators and indexing?

Difficulty:
Medium

```
-- JSONB operators
SELECT payload -> 'user' -> 'name'    -- navigate (returns jsonb)
SELECT payload ->> 'event_type'      -- text value
SELECT payload #> '{user,address,city}' -- path navigation
SELECT payload #>> '{user,name}'      -- path as text
```

```

SELECT payload @> '{"status": "active"}' -- contains
SELECT payload ? 'field_name'           -- key exists
SELECT payload ?| array['k1','k2']       -- any key exists
SELECT payload ?& array['k1','k2']       -- all keys exist
SELECT payload - 'password'              -- remove key

-- GIN index covers: @>, ?, ?|, ?&
CREATE INDEX idx ON events USING GIN (payload);

-- Path-specific index (smaller, faster for specific paths)
CREATE INDEX idx ON events ((payload ->> 'event_type'));
CREATE INDEX idx ON events ((payload -> 'user' ->> 'id'));

-- JSONB_PATH_QUERY (JSON Path Language)
SELECT jsonb_path_query(payload, '$.items[*].price ? (@ > 100)') FROM orders;

```

Q196. What is PostgreSQL temporary tables?

Difficulty:
Medium

```

-- Temporary tables: session-scoped, auto-dropped on disconnect
CREATE TEMP TABLE staging_data (
    id bigint, name text, amount numeric
);

-- Only visible in current session
-- Useful for: complex multi-step queries, import staging, calculations

-- UNLOGGED tables: faster writes, lost on crash
CREATE UNLOGGED TABLE cache_data (key text PRIMARY KEY, value text);
-- Use for: non-critical cache, session data, temp storage

-- CTE vs temp table:
-- CTE: inline, single query, auto-cleaned up
-- Temp table: persists within session, can have indexes
-- For complex multi-step transformations: temp table + indexes = faster

-- ON COMMIT behavior
CREATE TEMP TABLE t (id int) ON COMMIT DELETE ROWS; -- truncated each tx
CREATE TEMP TABLE t (id int) ON COMMIT DROP;         -- dropped each tx
CREATE TEMP TABLE t (id int) ON COMMIT PRESERVE ROWS; -- default

```

Q197. What is PostgreSQL materialized views?

Difficulty:
Medium

```

-- Materialized view: stored query result (like a cached view)
CREATE MATERIALIZED VIEW monthly_revenue AS
SELECT
    date_trunc('month', created_at) as month,
    sum(amount) as revenue,
    count(*) as order_count
FROM orders
WHERE status = 'completed'
GROUP BY 1;

CREATE UNIQUE INDEX ON monthly_revenue(month); -- needed for concurrent refresh

-- Refresh data

```

```

REFRESH MATERIALIZED VIEW monthly_revenue;           -- locks view during refresh
REFRESH MATERIALIZED VIEW CONCURRENTLY monthly_revenue; -- no lock (needs unique index)

-- Auto-refresh with pg_cron
SELECT cron.schedule('refresh-mv', '*/*10 * * * *',
    'REFRESH MATERIALIZED VIEW CONCURRENTLY monthly_revenue');

-- vs regular view:
-- View: re-executes query on each access (always fresh, can be slow)
-- Mat view: stored result (fast reads, stale until refresh)
-- Use mat view for: expensive aggregations, reporting queries, dashboards

```

Q198. What is PostgreSQL backup strategies?

Difficulty:
Medium

```

# pg_dump: logical backup (SQL or custom format)
pg_dump -Fc mydb > mydb.dump      # custom format (recommended)
pg_dump -Fd mydb -j 4 -f dumpdir  # directory format, 4 parallel jobs
pg_restore -d mydb mydb.dump
pg_restore -d mydb -j 4 mydb.dump  # parallel restore

# pg_basebackup: physical backup (faster for large DBs)
pg_basebackup -h localhost -U postgres -D /backup/base -Ft -z -P
# -Ft: tar format, -z: gzip, -P: progress

# PITR (Point-in-Time Recovery):
# 1. Base backup
# 2. WAL archiving (continuous)
# 3. Restore: base + WAL replay to target time

# Continuous archiving (WAL-G, popular)
WALG_S3_PREFIX=s3://my-backups/wal-g wal-g-pg backup-push /var/lib/postgresql/data
# Restore:
wal-g-pg backup-fetch /var/lib/postgresql/data LATEST
# Replay WAL to specific time via recovery.conf

# RDS/Aurora: automated backups built-in
# Retention: 1-35 days
# Point-in-time restore: 5-minute granularity

```

Q199. What is PostgreSQL extensions?

Difficulty:
Medium

```

SELECT * FROM pg_available_extensions ORDER BY name;

-- Popular extensions:
CREATE EXTENSION postgis;           -- geospatial
CREATE EXTENSION pg_trgm;           -- trigram similarity (fuzzy search)
CREATE EXTENSION uuid-ossdp;        -- UUID generation
CREATE EXTENSION hstore;            -- key-value pairs
CREATE EXTENSION ltree;             -- hierarchical tree paths
CREATE EXTENSION pg_stat_statements; -- query stats
CREATE EXTENSION pg_partman;        -- partition management
CREATE EXTENSION citus;             -- distributed PostgreSQL
CREATE EXTENSION vector;            -- vector similarity (pgvector)
CREATE EXTENSION timescaledb;       -- time-series optimization
CREATE EXTENSION pg_cron;           -- cron jobs inside PostgreSQL

```

```
CREATE EXTENSION plpgsql;          -- PL/pgSQL procedural language
CREATE EXTENSION pgtap;           -- testing framework

-- Check what's installed
SELECT * FROM pg_extension;
```

Q200. What is PostgreSQL for time-series data?

Difficulty:
Medium

```
-- TimescaleDB: PostgreSQL extension optimized for time-series
```

```
CREATE EXTENSION timescaledb;

-- Create hypertable (auto-partitioned by time)
CREATE TABLE metrics (
    time          TIMESTAMPTZ NOT NULL,
    device_id     TEXT,
    temperature   DOUBLE PRECISION
);
SELECT create_hypertable('metrics', 'time');

-- Continuous aggregates (auto-refreshed materialized views)
CREATE MATERIALIZED VIEW hourly_avg
WITH (timescaledb.continuous) AS
SELECT time_bucket('1 hour', time) AS bucket,
       device_id,
       avg(temperature) as avg_temp
FROM metrics
GROUP BY bucket, device_id;

SELECT add_continuous_aggregate_policy('hourly_avg',
    start_offset => INTERVAL '3 hours',
    end_offset   => INTERVAL '1 hour',
    schedule_interval => INTERVAL '1 hour');
```

```
-- Data retention
SELECT add_retention_policy('metrics', INTERVAL '90 days');
```

```
-- Compression (90%+ size reduction)
ALTER TABLE metrics SET (timescaledb.compress);
SELECT add_compression_policy('metrics', INTERVAL '7 days');
```

Q201. What is PostgreSQL production readiness checklist?

Difficulty:
Medium

```
-- Schema:
-- ■ Primary keys on all tables
-- ■ Foreign key constraints with proper ON DELETE behavior
-- ■ NOT NULL where appropriate
-- ■ Indexes on frequently queried columns (especially FK columns)
-- ■ Partial indexes for filtered queries
-- ■ Correct data types (bigint for IDs, timestamptz for times)

-- Performance:
-- ■ pg_stat_statements enabled and monitored
-- ■ Slow query log enabled (log_min_duration_statement = 1000)
-- ■ VACUUM/AUTOVACUUM running (monitor dead tuple ratio)
```

```
-- ■ Statistics up to date (ANALYZE regularly)
-- ■ Connection pooling (PgBouncer) in transaction mode

-- Reliability:
-- ■ Streaming replication to at least one replica
-- ■ Automated backups (pg_basebackup + WAL archiving)
-- ■ Backup restoration tested regularly
-- ■ PITR window meets RTO/RPO requirements
-- ■ Failover tested (Patroni or manual procedure)

-- Security:
-- ■ pg_hba.conf: no trust auth in production
-- ■ SSL enforced: ssl = on, ssl_cert_file configured
-- ■ Roles with least privilege
-- ■ Row-level security for multi-tenant
-- ■ pg_audit for compliance logging

-- Monitoring:
-- ■ pg_stat_replication lag alert
-- ■ Disk space alert (>80% usage)
-- ■ Transaction ID age alert (>1.5B)
-- ■ Deadlock frequency monitored
-- ■ Connection count vs max_connections
```